# Agentic Workflow Patterns

Author: Konrad Kowalczyk
Repository: ai-agents-JAVA-SPRING

# Agenda

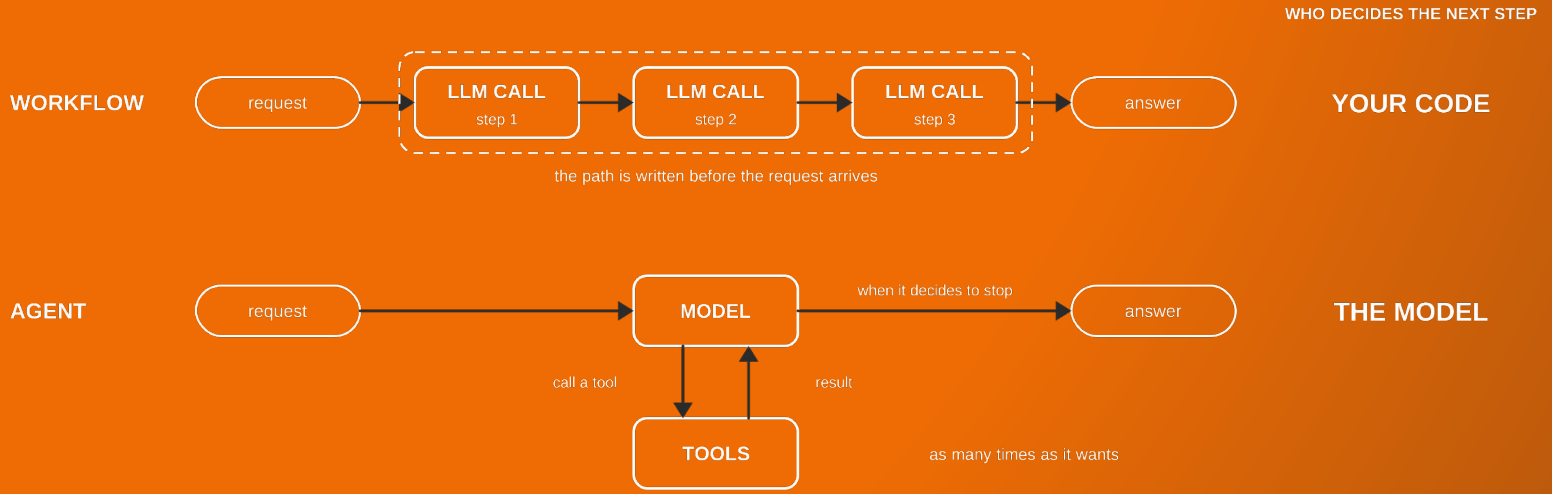
1. Workflow or agent?
2. Two modules, one domain
3. Five patterns
   - Prompt chaining
   - Routing + human in the loop
   - Parallelization
   - Evaluator-optimizer
   - Orchestrator-workers
4. Reading the traces
5. The DSL underneath



# Workflow or agent?

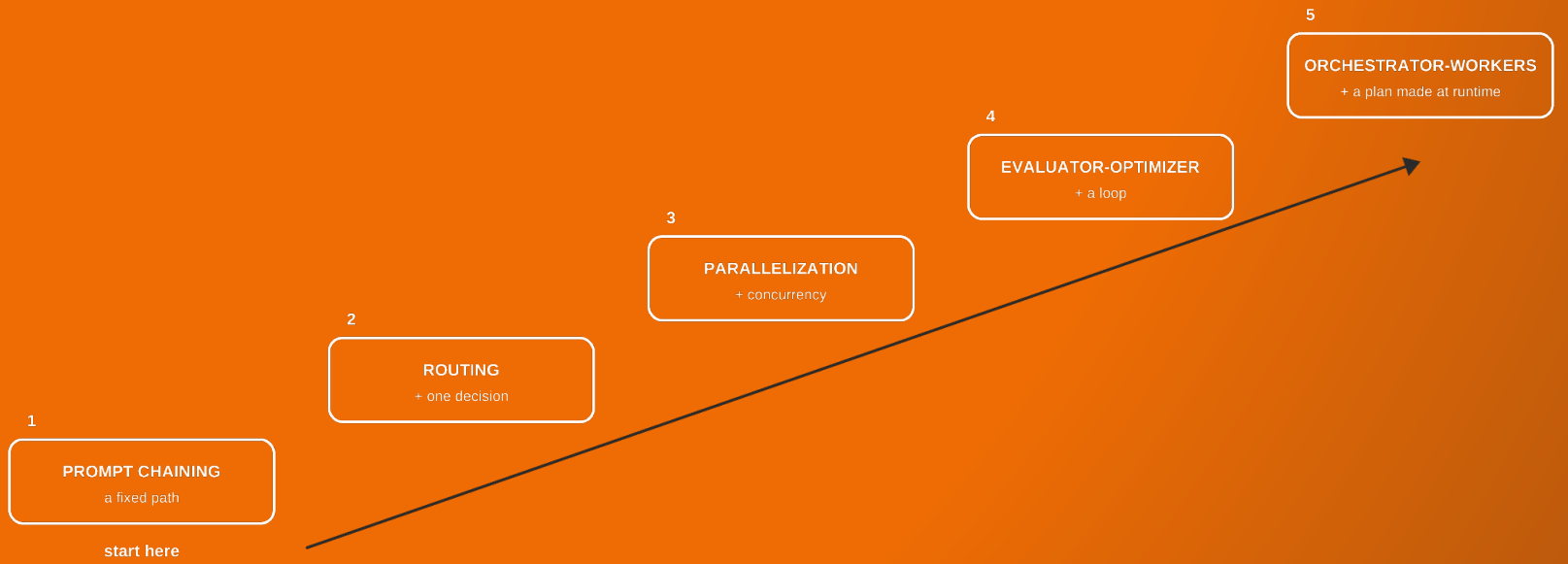
Who decides the next step?

- In a workflow, your code.
- In an agent, the model.



# Climb only when you must

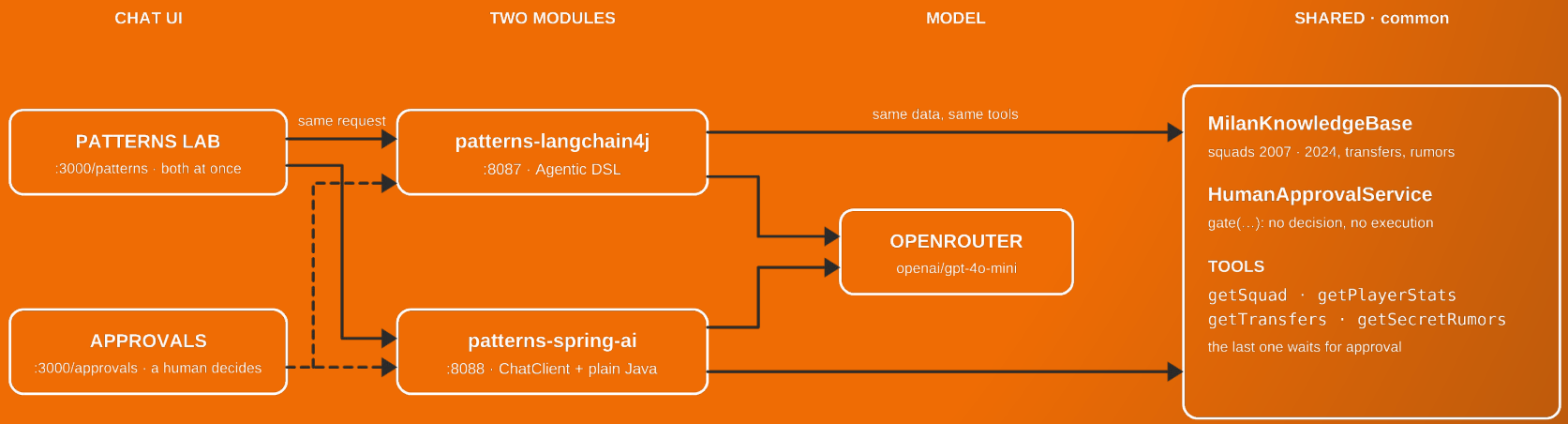
Find the simplest solution, and add complexity only when you need it.



# Two modules, one domain

Same five endpoints, same data, same four tools.

Only the framework changes.



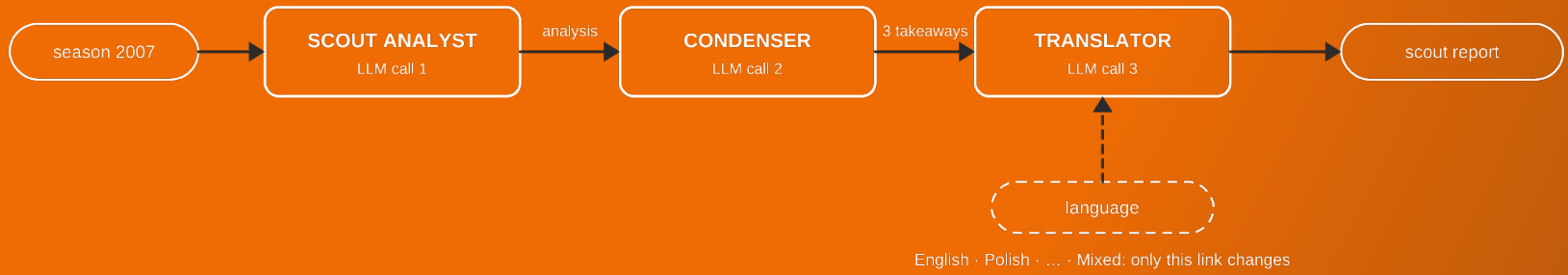
# Patterns Lab

**A page in the chat UI that sends the same request to LangChain4j and Spring AI at once.**



# Prompt chaining

**Break the task into steps.**
**Each step's answer is pasted into the next step's prompt.**
No shared history, every step is a fresh call.



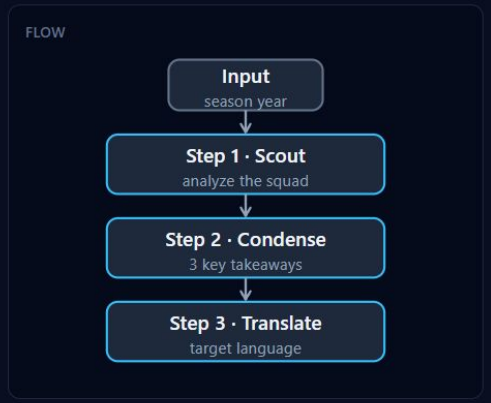
# Results

## 1. Prompt chaining

Scout analysis → condensed takeaways → report translated into the chosen language. Output of step N feeds step N+1.

2007 | Mixed (all languages) | ▷ Run both

Tempo: a staircase of sequential "chat" spans.




**LangChain4j** Agentic DSL — sequenceBuilder + outputKey — 13856 ms

- AC Milan ma zrównoważony skład z a strong defensive line, prowadzone przez światowej klasy obrońców Paolo Maldini și Alessandro Nesta, complemented by capabili full-backs.
- Mijlocul este ancorat de Andrea Pirlo, जिसकी creativity बहुत महत्वपूर्ण है, supported by tenacious oyuncular gibi Gennaro Gattuso और Clarence Seedorf, enhancing hem savunma hem de saldırı.
- Kaka выделяется как the team's ключевая атакующая угроза, with güçlü destek from clinical forvetler Filippo Inzaghi ve Alberto Gilardino, positioning AC Milan jako serious contenders in both domestic și European competitions.

**Spring AI** plain Java — three ChatClient calls — 11094 ms

- Zespół AC Milan z 2007 roku features a güçlü karışım of doświadczonych liderów, jak Maldini i Nesta, ensuring solid defensieve stabiliteit.
- Midfield maestro Andrea Pirlo चलाता creativity, supported by tenacious oyuncular like Gattuso and versatile yetenekler like Seedorf.
- Bir güçlü attacking trio led by Kaka, complemented by Inzaghi και Gilardino, highlights the team's ofensieve gücü and tactical intelligence.

# Prompt chaining → Spring AI

Plain Agent → no tools, no memory, the same system prompt on every step.

```
@Bean
ChatClient plainAgent(ChatClient.Builder builder) {    // the auto-configured builder
    return builder
            .defaultSystem("You are a concise football analyst. Answer briefly.")
            .build();                                  // no tools, no memory
}
```

# Three calls in a row

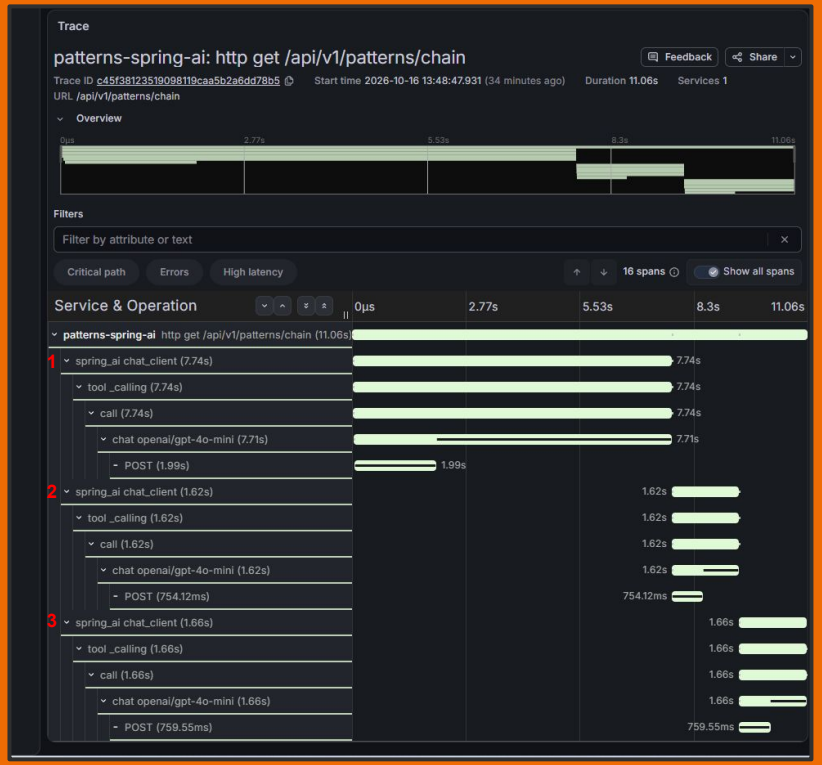
```
public String run(int seasonYear, String targetLanguage) {
    String language = TranslationLanguages.resolve(targetLanguage);  // "Mixed" → all languages
    var squad = MilanKnowledgeBase.squad(seasonYear);

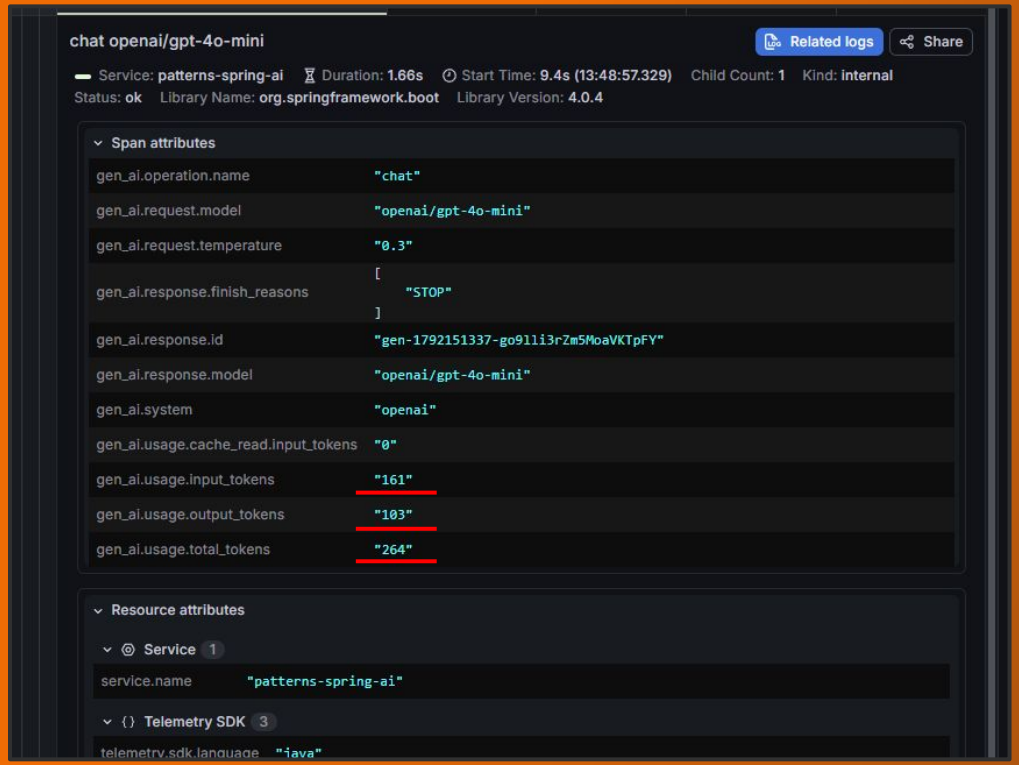
    // Step 1: the data is injected by code, not fetched by the model
    String analysis = plainAgent.prompt()
            .user("Write a short scout analysis of this AC Milan squad ("
                    + seasonYear + "): " + squad)
            .call().content();

    // Step 2: a validation gate could sit between the links
    String takeaways = plainAgent.prompt()
            .user("Condense this analysis into exactly 3 bullet takeaways:\n" + analysis)
            .call().content();

    // Step 3: render it in the requested language
    return plainAgent.prompt()
            .user("Translate the following into " + language
                    + ". Keep the bullet format:\n" + takeaways)
            .call().content();
}
```

# Trace


Trace
patterns-spring-ai: http get /api/v1/patterns/chain
Feedback
Share
Trace ID c45f381235190981l9caa5b2a6dd78b5
Start time 2026-10-16 13:48:47.931 (34 minutes ago)
Duration 11.06s
Services 1
URL /api/v1/patterns/chain
Overview
0µs
2.77s
5.53s
8.3s
11.06s
Filters
Filter by attribute or text
Critical path
Errors
High latency
16 spans
Show all spans
Service & Operation
0µs
2.77s
5.53s
8.3s
11.06s
patterns-spring-ai http get /api/v1/patterns/chain (11.06s)
1
spring_ai chat_client (7.74s)
7.74s
tool _calling (7.74s)
7.74s
call (7.74s)
7.74s
chat openai/gpt-4o-mini (7.71s)
7.71s
POST (1.99s)
1.99s
2
spring_ai chat_client (1.62s)
1.62s
tool _calling (1.62s)
1.62s
call (1.62s)
1.62s
chat openai/gpt-4o-mini (1.62s)
1.62s
POST (754.12ms)
754.12ms
3
spring_ai chat_client (1.66s)
1.66s
tool _calling (1.66s)
1.66s
call (1.66s)
1.66s
chat openai/gpt-4o-mini (1.66s)
1.66s
POST (759.55ms)
759.55ms


chat openai/gpt-4o-mini
Related logs
Share
Service: patterns-spring-ai
Duration: 1.66s
Start Time: 9.4s (13:48:57.329)
Child Count: 1
Kind: internal
Status: ok
Library Name: org.springframework.boot
Library Version: 4.0.4
Span attributes
gen_ai.operation.name "chat"
gen_ai.request.model "openai/gpt-4o-mini"
gen_ai.request.temperature "0.3"
gen_ai.response.finish_reasons [ "STOP" ]
gen_ai.response.id "gen-1792151337-go9lli3rZm5MoaVKTpFY"
gen_ai.response.model "openai/gpt-4o-mini"
gen_ai.system "openai"
gen_ai.usage.cache_read.input_tokens "0"
gen_ai.usage.input_tokens "161"
gen_ai.usage.output_tokens "103"
gen_ai.usage.total_tokens "264"
Resource attributes
Service 1
service.name "patterns-spring-ai"
Telemetry SDK 3
telemetry.sdk.language "java"

# Prompt chaining → LangChain4j

required

not required *

```java
public interface ScoutAnalyst {
    @Agent("Writes a scout analysis of a squad")
    @UserMessage("Write a short scout analysis of this AC Milan squad: {{squadData}}")
    String analyze(@V("squadData") String squadData);
}

public interface Condenser {
    @Agent("Condenses an analysis into takeaways")
    @UserMessage("Condense this analysis into exactly 3 bullet takeaways:\n{{analysis}}")
    String condense(@V("analysis") String analysis);           // what the analyst wrote
}

public interface Translator {
    @Agent("Renders takeaways in the requested language")
    @UserMessage("Translate the following into {{language}}. "
            + "Keep the bullet format:\n{{takeaways}}")
    String translate(@V("takeaways") String takeaways, @V("language") String language);
}

public interface ScoutReportWorkflow {                         // typed entry point
    @Agent("Produces a scout report for a squad in the requested language")
    String create(@V("squadData") String squadData, @V("language") String language);
}
```

# @Agent vs @SystemMessage

**@Agent** → read by whoever calls the agent. It says what this agent is for. A supervisor reads it to pick an agent, the way a model reads a `@Tool` description.

**@SystemMessage** → read by the model inside the agent, on every call. It says who to be and which rules to follow.

| Abstraction | Read by | Says |
|---|---|---|
| @Agent | the caller (a supervisor) | what this agent is for |
| @SystemMessage | the model, every call | who to be, which rules |
| @UserMessage | the model, every call | what to do with this input |

# The sequence is declared

```
public PromptChainingPattern(ChatModel chatModel) {
    var analyst = AgenticServices.agentBuilder(ScoutAnalyst.class)
            .chatModel(chatModel)
            .outputKey("analysis")                              // writes "analysis"
            .build();
    var condenser = AgenticServices.agentBuilder(Condenser.class)
            .chatModel(chatModel)
            .outputKey("takeaways")                             // writes "takeaways"
            .build();
    var translator = AgenticServices.agentBuilder(Translator.class)
            .chatModel(chatModel)
            .outputKey("translation")                           // writes "translation"
            .build();

    // The DSL: the sequence IS the pattern, no hand-written handoff code
    this.workflow = AgenticServices.sequenceBuilder(ScoutReportWorkflow.class)
            .subAgents(analyst, condenser, translator)
            .outputKey("translation")
            .build();
}
```

# AiServices vs AgenticServices

**AiServices** → the framework runs the loop. One interface, one AI service.

**AgenticServices** → you declare the workflow. A planner picks the next agent.

| | AISERVICES<br>`langchain4j` | AGENTICSERVICES<br>`langchain4j-agentic` |
|---|---|---|
| BUILDS | one AI service from an interface | agents, and workflows built from them |
| ONE CALL RUNS | one method call, tool loop included | the whole workflow, agent by agent |
| STEPS SHARE | your variables | AgenticScope<br>`outputKey → @V` |
| NEXT STEP | your Java code | a planner<br>`sequence · conditional · loop · parallel · supervisor` |
| IN THIS TALK | parallelization, orchestrator-workers | chaining, routing, evaluator-optimizer |

# One call starts the whole chain

```
public String run(int seasonYear, String targetLanguage) {
    // "Mixed" resolves to an instruction listing every supported language
    String language = TranslationLanguages.resolve(targetLanguage);

    var squad = MilanKnowledgeBase.squad(seasonYear);
    if (squad.isEmpty()) {
        return "No data for season " + seasonYear
                + ". Available: " + MilanKnowledgeBase.availableSeasons();
    }
    return workflow.create(squad.toString(), language);        // both enter the shared state
}
```

# Agentic Scope

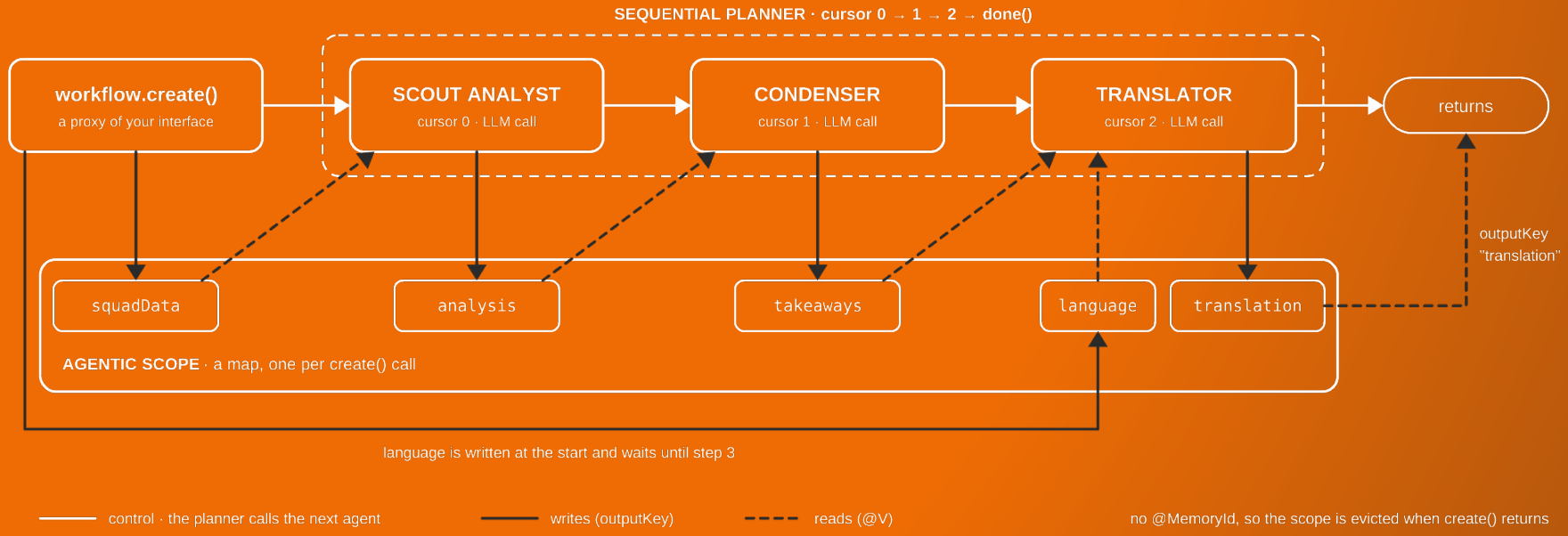
**AgenticScope → one shared map per workflow call**

- **writes** → the arguments, then each agent's outputKey
- **reads** → every @V, every condition, the final result
- **history** → every agent call, in order
- **lifetime** → one call, or kept per @MemoryId

**Keys are plain strings.**

**A typo fails at runtime.**

# DSL “sequence ”



# What the model actually sees

```
// trace b93bcd98b3d84edfea4b9906dcbe8bca · 3 × POST /api/v1/chat/completions

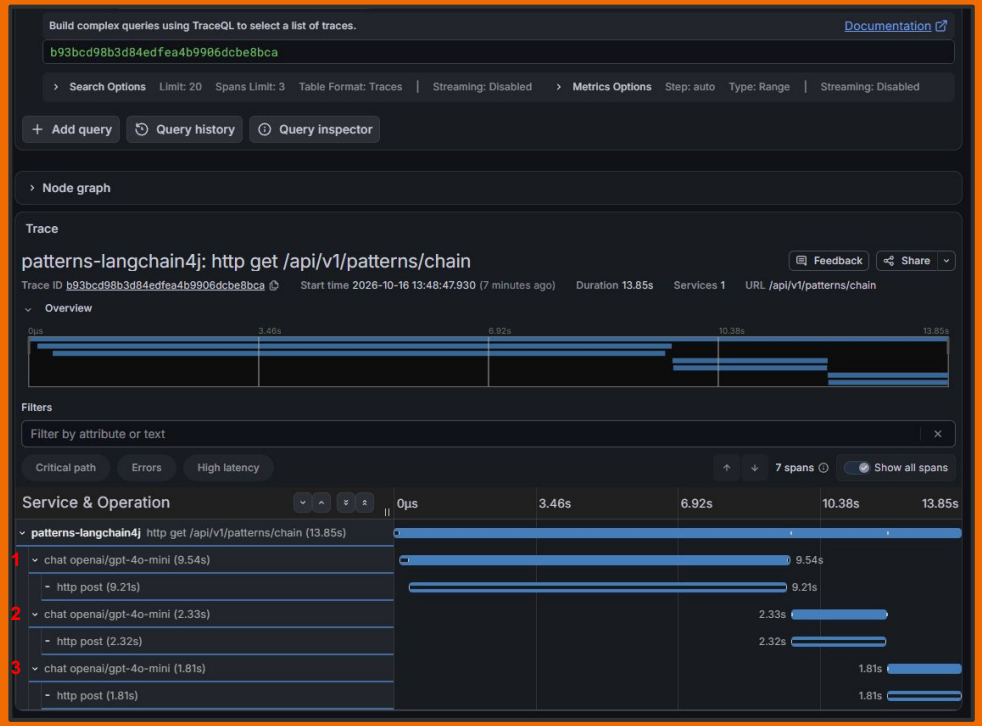
// 1 · SCOUT ANALYST · span 7f894e53… · 412 tokens in · 685 out · 9.3 s
"messages": [ { "role": "user",
    "content": "Write a short scout analysis of this AC Milan squad: [Player[name=Dida, …]]" } ]

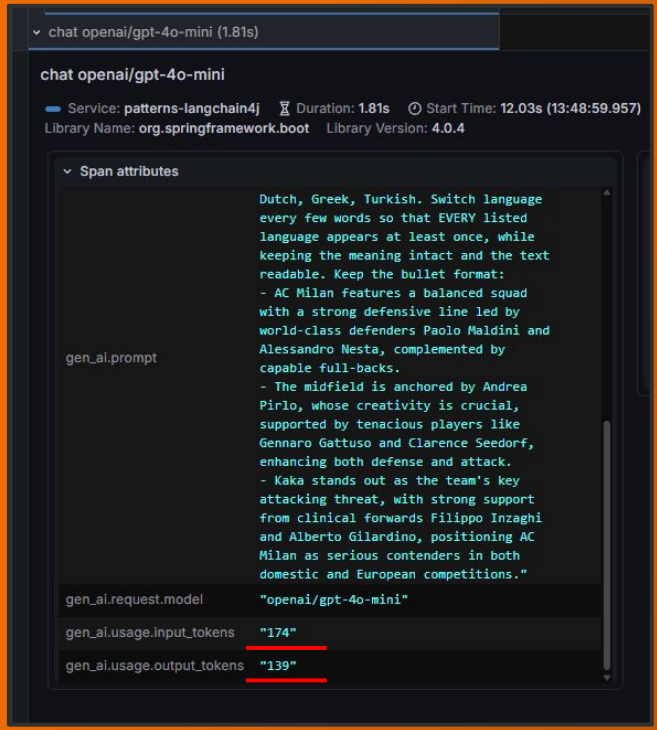
// 2 · CONDENSER · span 430b4a0f… · 704 tokens in · 111 out · 2.3 s
"messages": [ { "role": "user",
    "content": "Condense this analysis into exactly 3 bullet takeaways:\n### Scout Analysis …" } ]

// 3 · TRANSLATOR · span bc70932c… · 174 tokens in · 139 out · 1.8 s
"messages": [ { "role": "user",
    "content": "Translate the following into a deliberate MIX of all these languages: …" } ]
    // … "Keep the bullet format:\n" + the 3 takeaways

// one user message per call: no system message, no history, no @Agent text
```

# Trace

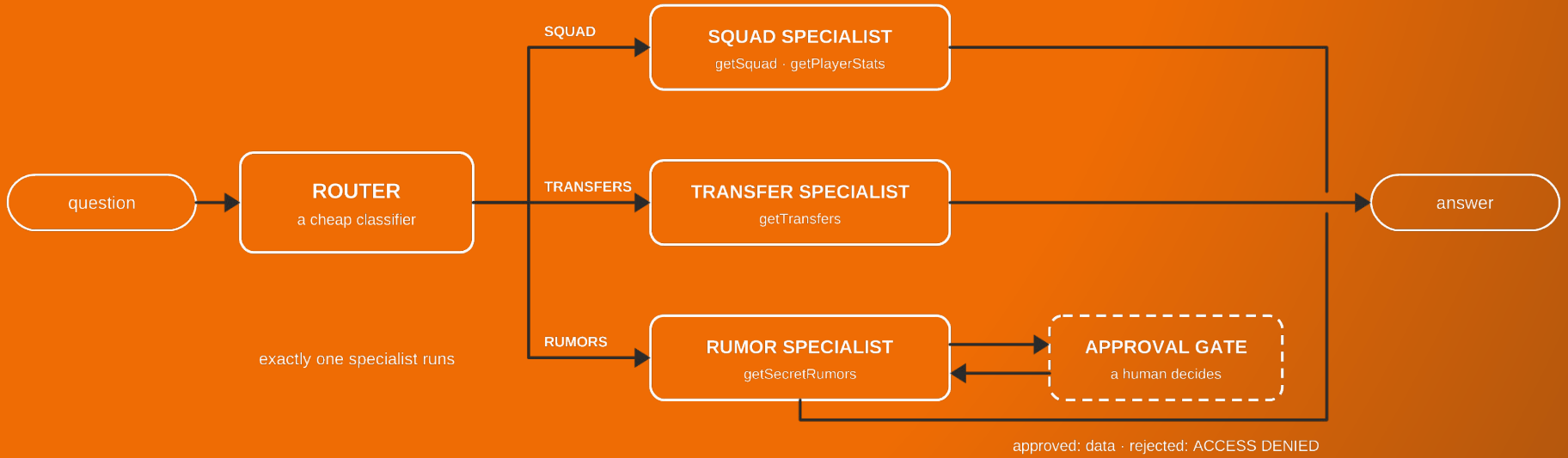

Build complex queries using TraceQL to select a list of traces.
Documentation
b93bcd98b3d84edfea4b9906dcbe8bca
Search Options Limit: 20 Spans Limit: 3 Table Format: Traces | Streaming: Disabled
Metrics Options Step: auto Type: Range | Streaming: Disabled
Add query
Query history
Query inspector
Node graph
Trace
patterns-langchain4j: http get /api/v1/patterns/chain
Feedback
Share
Trace ID b93bcd98b3d84edfea4b9906dcbe8bca
Start time 2026-10-16 13:48:47.930 (7 minutes ago)
Duration 13.85s
Services 1
URL /api/v1/patterns/chain
Overview
0µs
3.46s
6.92s
10.38s
13.85s
Filters
Filter by attribute or text
Critical path
Errors
High latency
7 spans
Show all spans
Service & Operation
0µs
3.46s
6.92s
10.38s
13.85s
patterns-langchain4j http get /api/v1/patterns/chain (13.85s)
1
chat openai/gpt-4o-mini (9.54s)
9.54s
http post (9.21s)
9.21s
2
chat openai/gpt-4o-mini (2.33s)
2.33s
http post (2.32s)
2.32s
3
chat openai/gpt-4o-mini (1.81s)
1.81s
http post (1.81s)
1.81s


chat openai/gpt-4o-mini (1.81s)
chat openai/gpt-4o-mini
Service: patterns-langchain4j
Duration: 1.81s
Start Time: 12.03s (13:48:59.957)
Library Name: org.springframework.boot
Library Version: 4.0.4
Span attributes
gen_ai.prompt
Dutch, Greek, Turkish. Switch language every few words so that EVERY listed language appears at least once, while keeping the meaning intact and the text readable. Keep the bullet format:
- AC Milan features a balanced squad with a strong defensive line led by world-class defenders Paolo Maldini and Alessandro Nesta, complemented by capable full-backs.
- The midfield is anchored by Andrea Pirlo, whose creativity is crucial, supported by tenacious players like Gennaro Gattuso and Clarence Seedorf, enhancing both defense and attack.
- Kaka stands out as the team's key attacking threat, with strong support from clinical forwards Filippo Inzaghi and Alberto Gilardino, positioning AC Milan as serious contenders in both domestic and European competitions."
gen_ai.request.model
"openai/gpt-4o-mini"
gen_ai.usage.input_tokens
"174"
gen_ai.usage.output_tokens
"139"

# Routing

A cheap classifier picks the specialist.

It recognizes, it never answers.



# Results

## 2. Routing

A cheap classifier picks the specialist: squad / transfers / rumors 🔒. The rumors branch calls a tool gated by human approval.

Show me the latest AC Milan transfer rumors.

▷ Run both

Tempo: one SHORT chat (router) + one LONG chat (specialist). Ask for rumors and the tool span waits for you in /approvals.




**LangChain4j** Agentic DSL — conditionalBuilder on shared state — 21390 ms

Here are the latest unconfirmed AC Milan transfer rumors:

1. **Jonathan David** - Linked to AC Milan with a probability of 65%. The agent met the sporting director twice in Casa Milan last week.

2. **Rafael Leao** - Linked to Al-Hilal with a probability of 20%. A Saudi bid of €90M was rejected; the player wants to play Champions League football.

3. **Joshua Zirkzee** - Linked to AC Milan with a probability of 45%. His release clause expires at the end of June, and the board is split on the fee.

**Spring AI** plain Java — switch on an enum — 21837 ms

Here are the latest AC Milan transfer rumors (unconfirmed):

1. **Jonathan David** - Linked to AC Milan with a probability of 65%. Note: The agent met the sporting director twice in Casa Milan last week.

2. **Rafael Leao** - Linked to Al-Hilal with a probability of 20%. Note: A Saudi bid of €90M was rejected; the player wants Champions League football.

3. **Joshua Zirkzee** - Linked to AC Milan with a probability of 45%. Note: His release clause expires at the end of June; the board is split on the fee.

# Routing → Spring AI

**Two clients, two jobs**

**Plain Agent** → has no tools, so it can only recognize.

**Milan Agent** → has all four tools and answers.

```
@Bean                                                                   // the router
ChatClient plainAgent(ChatClient.Builder builder) {
    return builder
            .defaultSystem("You are a concise football analyst. Answer briefly.")
            .build();                                                   // no tools
}

@Bean                                                                   // the specialists
ChatClient milanAgent(ChatClient.Builder builder, MilanTools milanTools) {
    return builder
            .defaultSystem("You are an AC Milan data analyst. …")       // replaced per specialist
            .defaultTools(milanTools)                                   // all four tools
            .build();
}

public enum Route { SQUAD, TRANSFERS, RUMORS }
public record RouteChoice(Route route) {}                               // a record, not a bare enum
```

# Classify, with a fallback

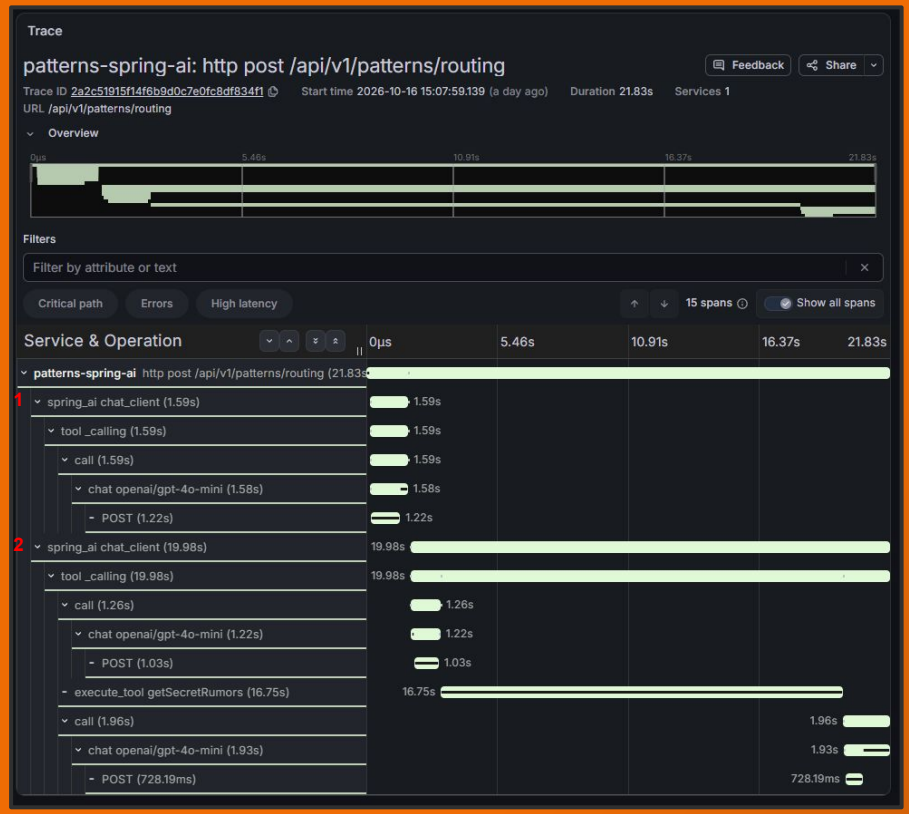
```
private Route classify(String question) {
    String prompt = "Classify this AC Milan question into exactly one category "
            + "(SQUAD = players/lineups, TRANSFERS = confirmed moves/fees, "
            + "RUMORS = gossip/speculation): " + question;
    try {
        RouteChoice choice = plainAgent.prompt()
                .user(prompt)
                .call()
                .entity(RouteChoice.class);                    // structured output
        if (choice != null && choice.route() != null) {
            return choice.route();
        }
    } catch (Exception e) {
        log.warn("[ROUTING] structured classification failed ({}), "
                + "falling back to string parsing", e.toString());
    }
    String raw = plainAgent.prompt()                           // fallback: plain text
            .user(prompt + " Answer with ONLY the category name.")
            .call().content();
    String normalized = raw == null ? "" : raw.toUpperCase();
    if (normalized.contains("TRANSFER")) return Route.TRANSFERS;
    if (normalized.contains("RUMOR")) return Route.RUMORS;
    return Route.SQUAD;                                        // a default branch
}
```

# The dispatch is a switch

```
public String run(String question) {
    Route route = classify(question);
    log.info("[ROUTING] '{}' -> {}", question, route);

    return switch (route) {                                    // the dispatch is plain Java
        case SQUAD -> milanAgent.prompt()
                .system("You are a squad specialist. ALWAYS call getSquad …")
                .user(question)
                .call().content();
        case TRANSFERS -> milanAgent.prompt()
                .system("You are a transfer-market specialist. ALWAYS call getTransfers …")
                .user(question)
                .call().content();
        case RUMORS -> milanAgent.prompt()
                .system("You are an insider on AC Milan transfer rumors. ALWAYS call …")
                .user(question)
                .call().content();
    };
}
```

# Trace


Trace
patterns-spring-ai: http post /api/v1/patterns/routing
Feedback
Share
Trace ID 2a2c51915f14f6b9d0c7e0fc8df834f1
Start time 2026-10-16 15:07:59.139 (a day ago)
Duration 21.83s
Services 1
URL /api/v1/patterns/routing
Overview
0µs
5.46s
10.91s
16.37s
21.83s
Filters
Filter by attribute or text
Critical path
Errors
High latency
15 spans
Show all spans
Service & Operation
patterns-spring-ai http post /api/v1/patterns/routing (21.83s
1
spring_ai chat_client (1.59s)
1.59s
tool _calling (1.59s)
1.59s
call (1.59s)
1.59s
chat openai/gpt-4o-mini (1.58s)
1.58s
POST (1.22s)
1.22s
2
spring_ai chat_client (19.98s)
19.98s
tool _calling (19.98s)
19.98s
call (1.26s)
1.26s
chat openai/gpt-4o-mini (1.22s)
1.22s
POST (1.03s)
1.03s
execute_tool getSecretRumors (16.75s)
16.75s
call (1.96s)
1.96s
chat openai/gpt-4o-mini (1.93s)
1.93s
POST (728.19ms)
728.19ms

# Routing → LangChain4j

**One interface per role**
The router returns an enum.
Each specialist adds a @SystemMessage.

```
public interface Router {
    @Agent("Classifies an AC Milan question into one specialist category")
    @UserMessage("Classify this AC Milan question into exactly one category "
            + "(SQUAD = players/lineups, TRANSFERS = confirmed moves/fees, "
            + "RUMORS = gossip/speculation): {{question}}")
    Route classify(@V("question") String question);           // returns the enum
}

public interface TransferSpecialist {
    @Agent("Answers confirmed transfer questions")
    @SystemMessage("You are a transfer-market specialist. ALWAYS call "
            + "getTransfers BEFORE answering. Never ask whether you should "
            + "check — check first, then answer with the data.")
    @UserMessage("{{question}}")
    String answer(@V("question") String question);
}

// SquadSpecialist, RumorSpecialist: the same shape, their own prompt
```

# Every specialist writes the same key

```
public RoutingPattern(ChatModel chatModel, ToolProvider milanToolProvider) {

    var router = AgenticServices.agentBuilder(Router.class)
            .chatModel(chatModel)
            .outputKey(ROUTE_KEY)
            .build();

    // Every specialist writes to the SAME key: exactly one of them runs,
    // so the workflow's output key is unambiguous.
    var squad = AgenticServices.agentBuilder(SquadSpecialist.class)
            .chatModel(chatModel)
            .toolProvider(milanToolProvider)
            .outputKey(ANSWER_KEY)
            .build();
    var transfers = AgenticServices.agentBuilder(TransferSpecialist.class)
            .chatModel(chatModel)
            .toolProvider(milanToolProvider)
            .outputKey(ANSWER_KEY)
            .build();
    var rumors = AgenticServices.agentBuilder(RumorSpecialist.class)
            .chatModel(chatModel)
            .toolProvider(milanToolProvider)
            .outputKey(ANSWER_KEY)
            .build();
```

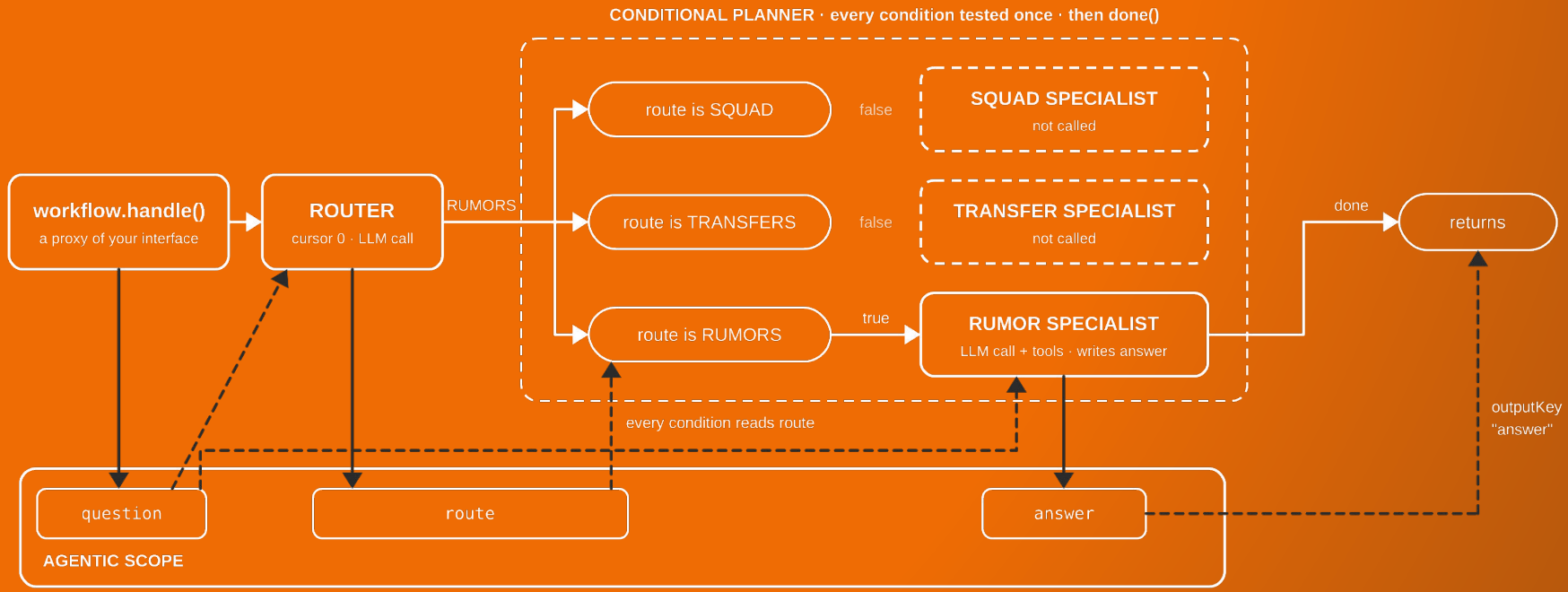
# The branch is declared

```
    var dispatch = AgenticServices.conditionalBuilder()
            .subAgents("route is SQUAD", routeIs(Route.SQUAD), squad)
            .subAgents("route is TRANSFERS", routeIs(Route.TRANSFERS), transfers)
            .subAgents("route is RUMORS", routeIs(Route.RUMORS), rumors)
            .outputKey(ANSWER_KEY)
            .build();

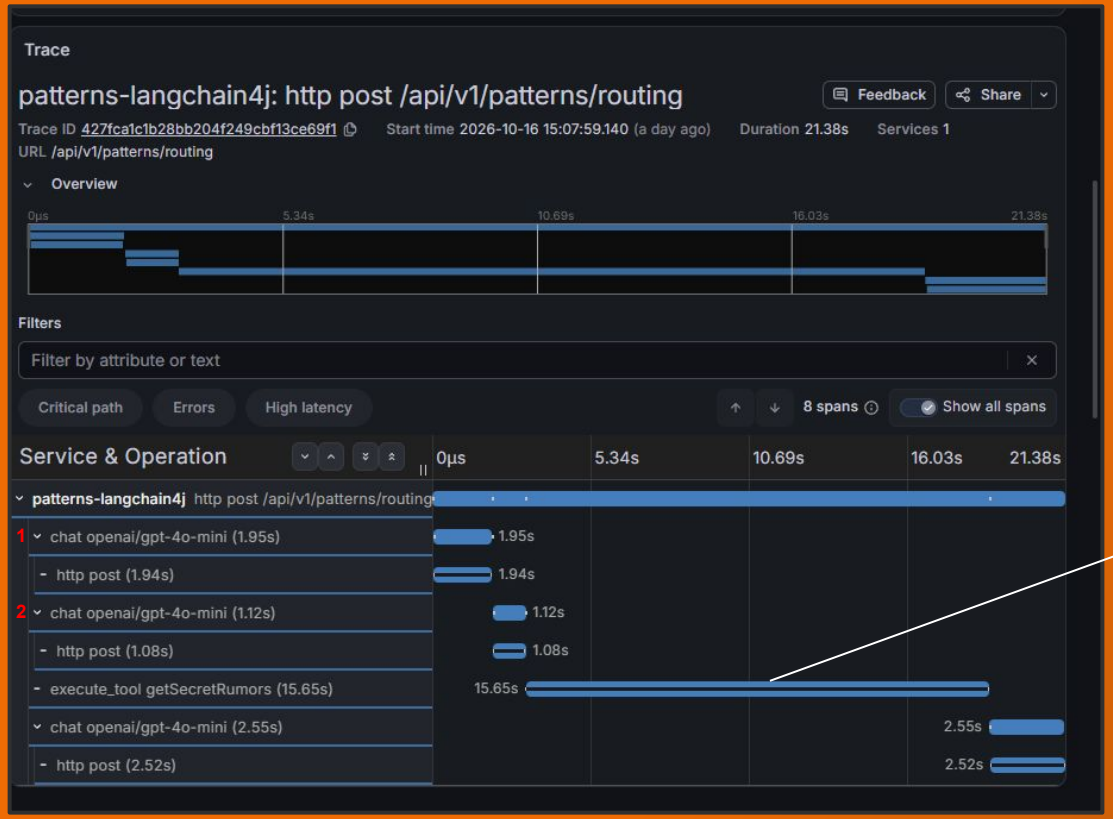
    this.workflow = AgenticServices.sequenceBuilder(RoutedAnswerWorkflow.class)
            .subAgents(router, dispatch)
            .outputKey(ANSWER_KEY)
            .build();
}
```

```
private static Predicate<AgenticScope> routeIs(Route route) {
    return scope -> route.name()
            .equals(String.valueOf(scope.readState(ROUTE_KEY, "")).trim());
}
```

# DSL “conditional”



# Trace




**Why does it take so long?**

# Approval flow

Not a slow model.
A slow human.

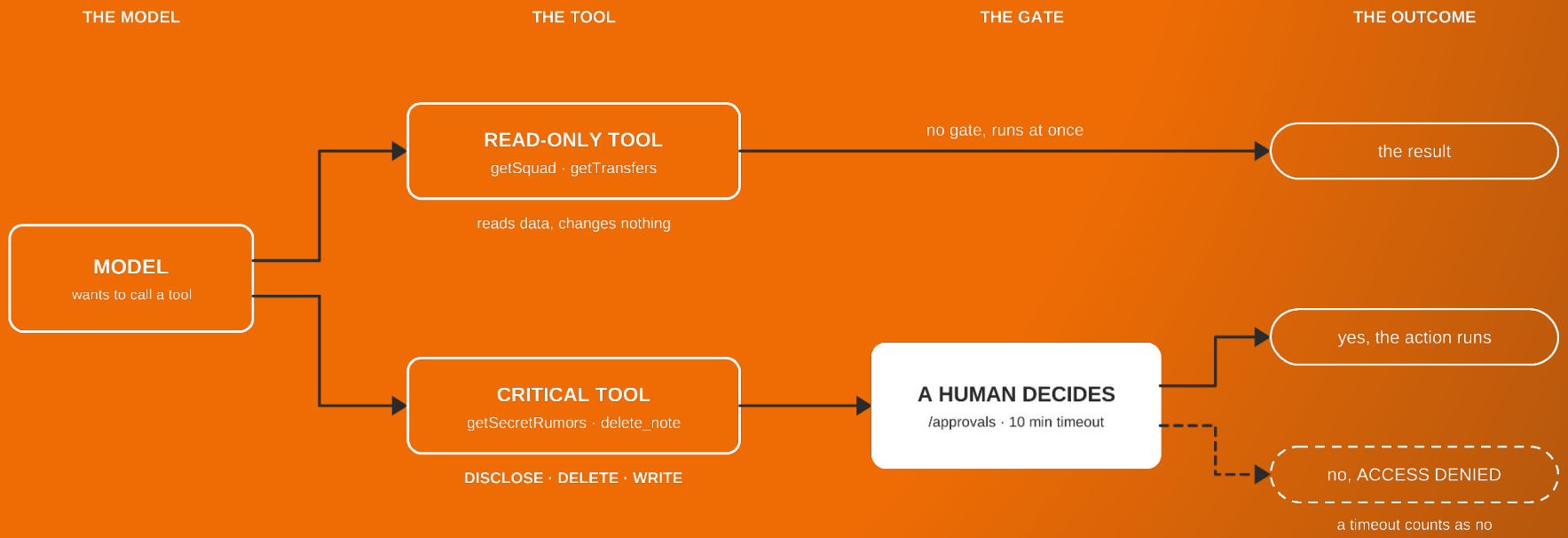
but…

**Some decisions are worth waiting for.**

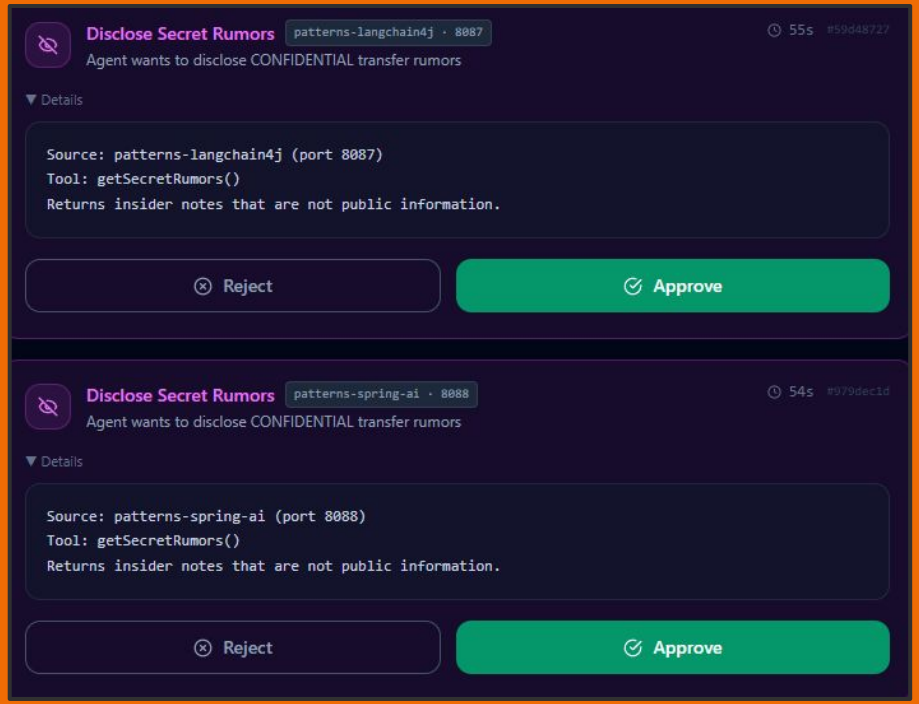


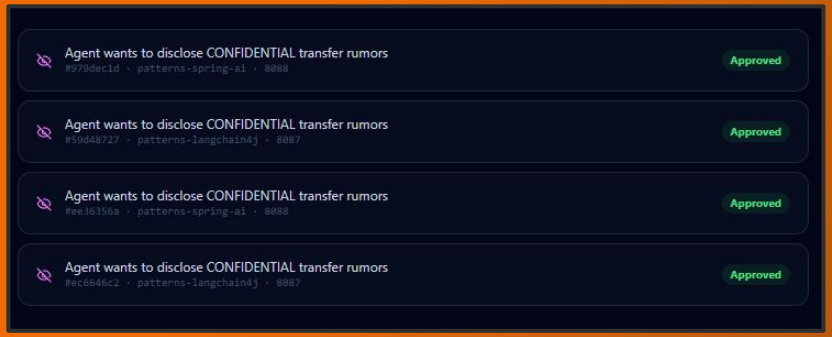
# Human in the Loop

## The model can ask, but only a human can approve.

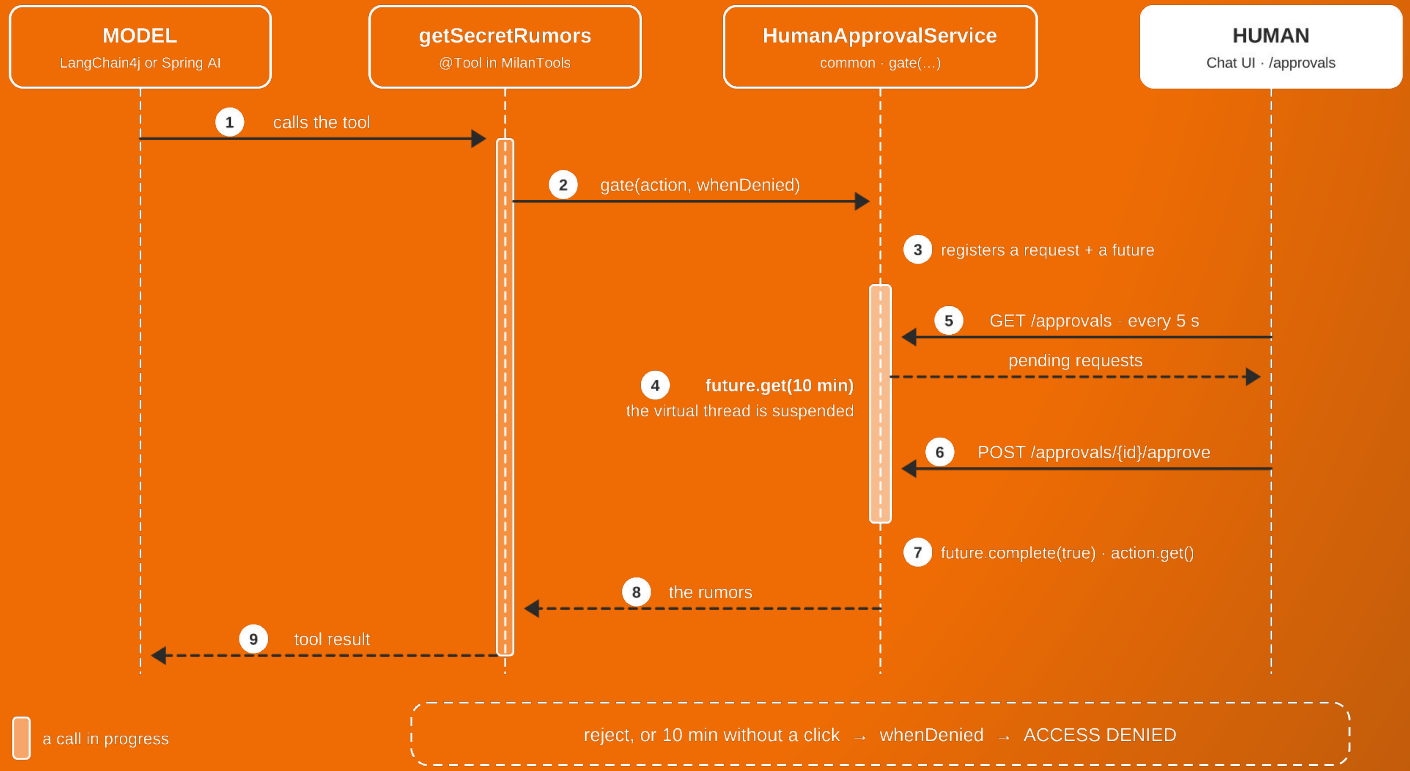


# Critical operations need a human to review and approve them.



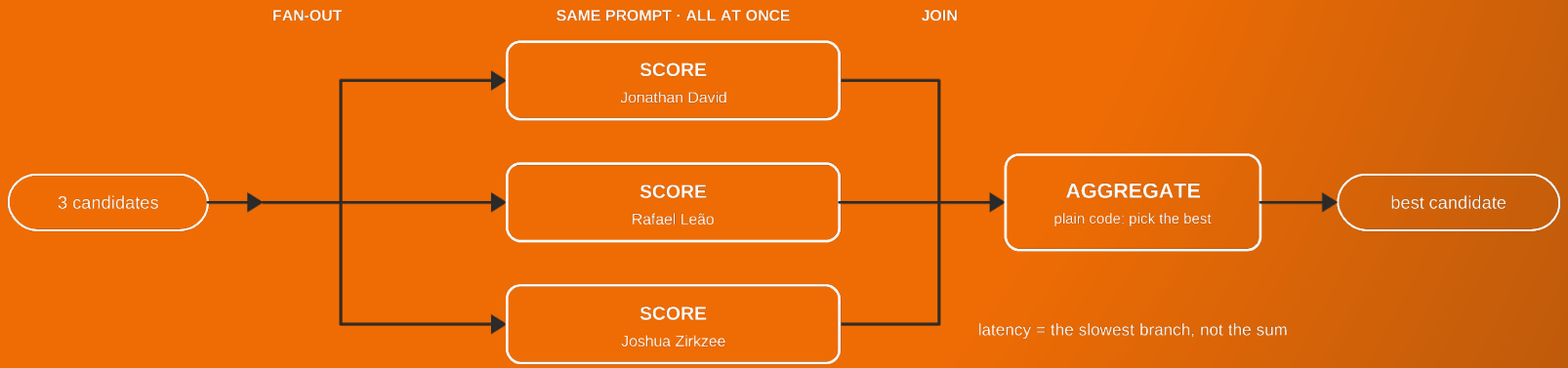
# Schema


MODEL
LangChain4j or Spring AI
getSecretRumors
@Tool in MilanTools
HumanApprovalService
common · gate(...)
HUMAN
Chat UI · /approvals
1 calls the tool
2 gate(action, whenDenied)
3 registers a request + a future
5 GET /approvals · every 5 s
pending requests
4 future.get(10 min)
the virtual thread is suspended
6 POST /approvals/{id}/approve
7 future.complete(true) · action.get()
8 the rumors
9 tool result
a call in progress
reject, or 10 min without a click → whenDenied → ACCESS DENIED

# Parallelization

Every candidate is scored at the same time.

The code waits for all of them and picks the best.



# Results

### 3. Parallelization

Scores all rumor candidates concurrently, then aggregates.

▷ Run both

Tempo: OVERLAPPING chat spans — the waterfall stops being a staircase.




**LangChain4j** plain Java — CompletableFuture (DSL adds nothing here) `2889 ms`

Best candidate: Jonathan David (score 0.75) — With a 65% probability of transfer and recent meetings between the agent and AC Milan's sporting director, there is a solid chance of this transfer materializing. Additionally, AC Milan could benefit from David's attacking prowess, addressing their squad needs for a forward.

All scores: [CandidateScore[player=Jonathan David, score=0.75, rationale=With a 65% probability of transfer and recent meetings between the agent and AC Milan's sporting director, there is a solid chance of this transfer materializing. Additionally, AC Milan could benefit from David's attacking prowess, addressing their squad needs for a forward.], CandidateScore[player=Rafael Leao, score=0.6, rationale=Leao is a key player for AC Milan, and the reported interest from Al-Hilal indicates a significant bid. However, the 20% probability of the transfer happening suggests that it is unlikely. Additionally, Leao's desire for Champions

**Spring AI** plain Java — CompletableFuture `2373 ms`

Best candidate: Jonathan David (score 0.8) — Strong probability and fits squad needs.

All scores: [CandidateScore[player=Jonathan David, score=0.8, rationale=Strong probability and fits squad needs.], CandidateScore[player=Rafael Leao, score=0.3, rationale=Low probability of transfer and player preference for Champions League football.], CandidateScore[player=Joshua Zirkzee, score=0.6, rationale=Moderate probability and squad needs considered, but board split on fee affects viability.]]

# It’s quite simple

**Both sides are plain Java + a model call.**

**LangChain4j**

```java
public record CandidateScore(String player, double score, String rationale) {}

interface CandidateScorer {
    @UserMessage("Score this transfer candidate for AC Milan from 0.0 to 1.0 "
            + "considering probability and squad needs. Candidate: {{it}}")
    CandidateScore score(String rumor);
}

private final CandidateScorer scorer;
private final ExecutorService executor =
        Executors.newVirtualThreadPerTaskExecutor();

public ParallelizationPattern(ChatModel chatModel) {
    this.scorer = AiServices.builder(CandidateScorer.class)
            .chatModel(chatModel)
            .build();
}
```

```java
public String run() {
    var candidates = MilanKnowledgeBase.secretRumors();
    log.info("[PARALLEL] fan-out over {} candidates", candidates.size());

    List<CompletableFuture<CandidateScore>> futures = candidates.stream()
            .map(rumor -> CompletableFuture.supplyAsync(
                    () -> scorer.score(rumor.toString()), executor))
            .toList();

    List<CandidateScore> scores = futures.stream()
            .map(CompletableFuture::join)
            .toList();
    scores.forEach(s -> log.info("[PARALLEL] {} -> {}", s.player(), s.score()));

    CandidateScore best = scores.stream()
            .max(Comparator.comparingDouble(CandidateScore::score))
            .orElseThrow();

    return "Best candidate: " + best.player()
            + " (score " + best.score() + ") — " + best.rationale()
            + "\n\nAll scores: " + scores;
}
```

# Let the model fill a record, let Java do the math

**Ask the model for a record, not for text.**

A model call costs time, money and a chance of a wrong answer.

Arithmetic costs none of them.

```java
public record CandidateScore(String player, Double score, String rationale) {

    double scoreOrZero() {
        return score == null ? 0.0 : score;
    }

    boolean isUsable() {
        return player != null && !player.isBlank() && score != null;
    }
}
```

```
"user: Score this transfer candidate for AC Milan from 0.0
to 1.0 considering probability and squad needs. Candidate:
Rumor[player=Rafael Leao, linkedClub=Al-Hilal,
probabilityPercent=20, secretNote=Saudi bid of 90M rejected;
player wants Champions League football.]
You must answer strictly in the following JSON format: {
"player": (type: string),
"score": (type: double),
"rationale": (type: string)
}"
```

# Spring AI entity

```
CandidateScore parsed = plainAgent.prompt()
        .user(prompt(rumor, attempt))
        .call()
        .entity(CandidateScore.class);
```

```
> 2026-11-02 21:00:57.929  INFO  Chat Model Prompt Content:
["You are a concise football analyst. Answer briefly.", "Score this transfer candidate
for AC Milan from 0.0 to 1.0 considering probability and squad needs. Candidate: Rumor
[player=Jonathan David, linkedClub=AC Milan, probabilityPercent=65, secretNote=Agent me
t the sporting director twice in Casa Milan last week.]
Your response should be in JSON format.
Do not include any explanations, only provide a RFC8259 compliant JSON response followi
ng this format without deviation.
Do not include markdown code blocks in your response.
Remove the ```json markdown from the output.
Here is the JSON Schema instance your output must adhere to:
```{
  "$schema" : "https://json-schema.org/draft/2020-12/schema",
  "type" : "object",
  "properties" : {
    "player" : {
      "type" : "string"
    },
    "rationale" : {
      "type" : "string"
    },
    "score" : {
      "type" : "number",
      "format" : "double"
    }
  },
  "required" : [ "player", "rationale", "score" ],
  "additionalProperties" : false
}```
"]
```
```

# DSL… but why?

Agents and Agentic AI | LangChain4j

- parallelBuilder
- parallelMapperBuilder

**Parallelization has no state and no condition to declare.**

**CompletableFuture does the same job and hides less.**

**The DSL can catch a crash. It can’t spot a wrong answer.**

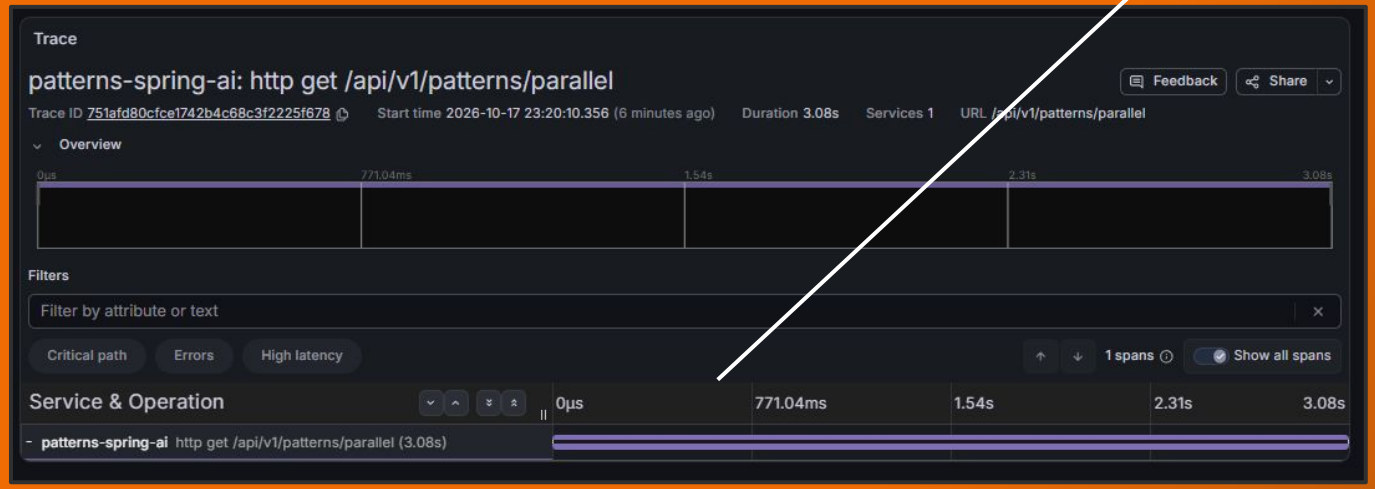
```
CandidateScorer scorer = AgenticServices
        .agentBuilder(CandidateScorer.class)
        .chatModel(chatModel)
        .outputKey("score")
        .build();

BatchScoringAgent agent = AgenticServices
        .parallelMapperBuilder(BatchScoringAgent.class)
        .subAgents(scorer)
        .itemsProvider("rumors")
        .executor(Executors.newVirtualThreadPerTaskExecutor())
        .build();

List<Rumor> rumors = MilanKnowledgeBase.secretRumors();
List<CandidateScore> scores = agent.scoreAll(rumors);
```

# More Threads = More problems

Should be 3



# The Problem

**The trace context lives in a ThreadLocal.**
A new thread starts without it, so every branch opens its own trace.

```
private final ExecutorService executor =
        Executors.newVirtualThreadPerTaskExecutor();

// every branch starts on a new thread, with an empty context
List<CompletableFuture<CandidateScore>> futures = candidates.stream()
        .map(rumor -> CompletableFuture.supplyAsync(() -> score(rumor), executor))
        .toList();
```

# The Solution

**Wrap the executor.**

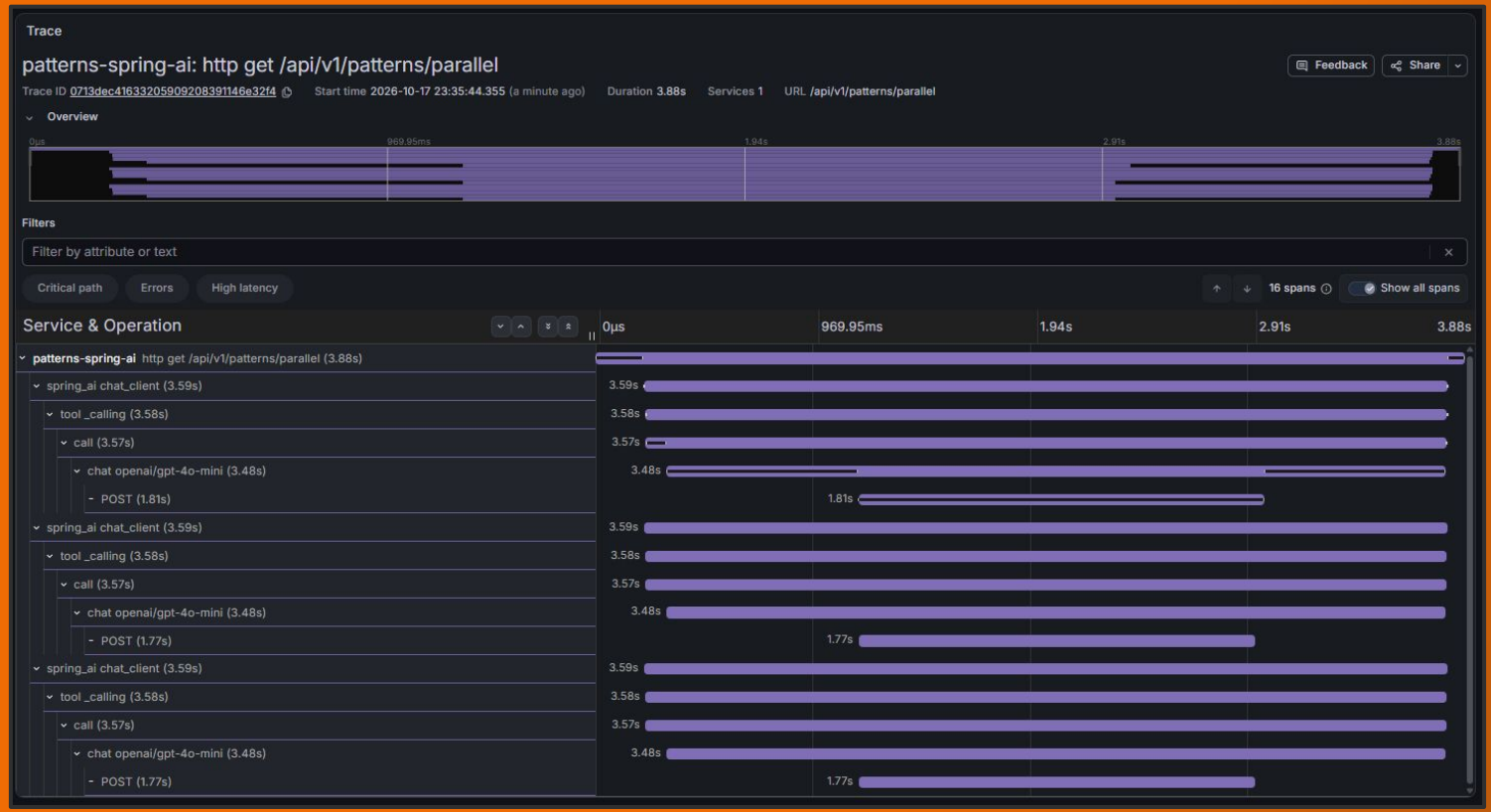
**It copies the context into every task.**

```
import io.micrometer.context.ContextExecutorService;
import io.micrometer.context.ContextSnapshotFactory;

private final ExecutorService executor = ContextExecutorService.wrap(
        Executors.newVirtualThreadPerTaskExecutor(),
        ContextSnapshotFactory.builder().build());

// supplyAsync(() -> score(rumor), executor) stays exactly the same
```

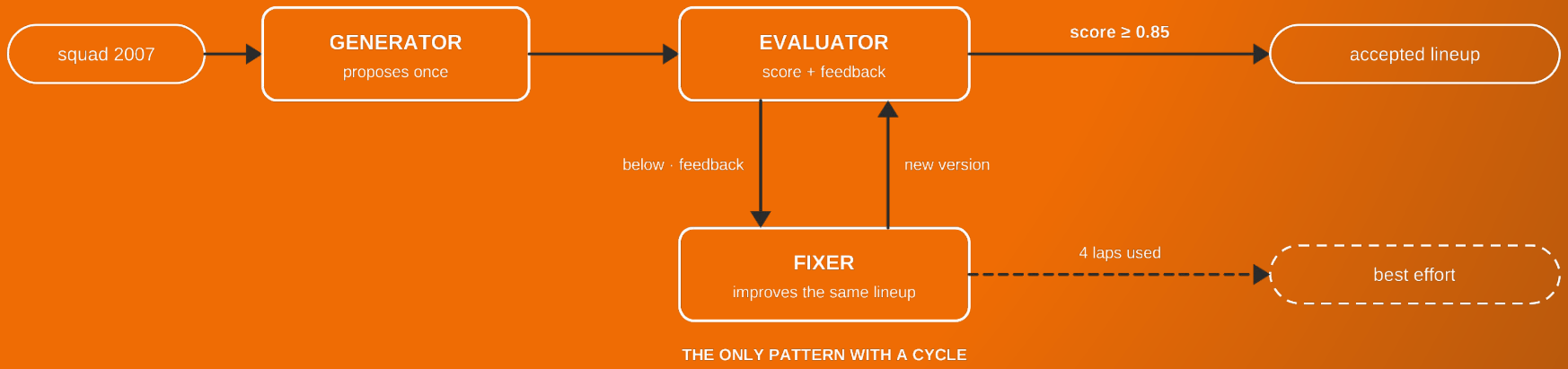
# Fixed Trace


Trace
patterns-spring-ai: http get /api/v1/patterns/parallel
Feedback
Share
Trace ID 0713dec4163320590920839l146e32f4
Start time 2026-10-17 23:35:44.355 (a minute ago)
Duration 3.88s
Services 1
URL /api/v1/patterns/parallel
Overview
0µs
969.95ms
1.94s
2.91s
3.88s
Filters
Filter by attribute or text
Critical path
Errors
High latency
16 spans
Show all spans
Service & Operation
0µs
969.95ms
1.94s
2.91s
3.88s
patterns-spring-ai http get /api/v1/patterns/parallel (3.88s)
spring_ai chat_client (3.59s)
tool _calling (3.58s)
call (3.57s)
chat openai/gpt-4o-mini (3.48s)
POST (1.81s)
spring_ai chat_client (3.59s)
tool _calling (3.58s)
call (3.57s)
chat openai/gpt-4o-mini (3.48s)
POST (1.77s)
spring_ai chat_client (3.59s)
tool _calling (3.58s)
call (3.57s)
chat openai/gpt-4o-mini (3.48s)
POST (1.77s)
3.59s
3.58s
3.57s
3.48s
1.81s
3.59s
3.58s
3.57s
3.48s
1.77s
3.59s
3.58s
3.57s
3.48s
1.77s

# Evaluator-optimizer

One call proposes, one scores, one fixes.

Repeat until the score is good enough or the laps run out.



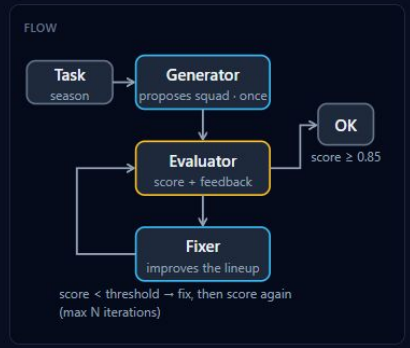
# Results

## 4. Evaluator-optimizer

Generator proposes a squad once, then a scorer/fixer loop improves it until score ≥ 0.85.

2007 ▷ Run both

Tempo: one chat, then N repetitions of a score+fix pair.




**LangChain4j** Agentic DSL — loopBuilder + exitCondition — 8016 ms

Based on the players provided, I propose a 4-3-1-2 formation for AC Milan. This formation allows for a solid defense, a strong midfield, and two forwards supported by an attacking midfielder. Here's the starting XI:

### Formation: 4-3-1-2

1. **Goalkeeper:**
   - Dida (Shirt Number: 1)

2. **Defenders:**
   - Right Back: Massimo Oddo (Shirt Number: 44)
   - Center Back: Paolo Maldini (Shirt Number: 3)
   - Center Back: Alessandro Nesta (Shirt Number: 13)

**Spring AI** plain Java — for loop + if (score >= threshold) — 4863 ms

Accepted after 1 iteration(s) (score 0.9):
**Formation: 4-3-1-2**

**Starting XI:**

- **GK:** Dida (1)
- **RB:** Cafu (2)
- **CB:** Paolo Maldini (3)
- **CB:** Alessandro Nesta (13)
- **LB:** Marek Jankulovski (18)
- **CM:** Gennaro Gattuso (8)
- **CM:** Andrea Pirlo (21)
- **CM:** Massimo Ambrosini (23)

# Evaluator-optimizer → Spring AI

The **generator** runs once, outside the loop.

```
public record Evaluation(double score, String feedback) {}

private static final int MAX_ITERATIONS = 4;
private static final double THRESHOLD = 0.85;

public String run(int seasonYear) {
    var squad = MilanKnowledgeBase.squad(seasonYear);
    if (squad.isEmpty()) {
        return "No data for season " + seasonYear;
    }
    log.info("[EVALUATOR] season={} threshold={} — starting review loop",
            seasonYear, THRESHOLD);

    // Generator — runs ONCE, outside the loop (mirrors the LC4j proposer)
    String solution = plainAgent.prompt()
            .user("Propose a starting XI formation and lineup for AC Milan "
                    + seasonYear + " using ONLY players from this list "
                    + "(name and shirt number): " + squad)
            .call()
            .content();
```

# Main Loop

```
    for (int i = 1; i <= MAX_ITERATIONS; i++) {
        // One span per lap. The two chat spans of this iteration nest
        // inside it, so the waterfall groups the pair instead of showing
        // a flat run of chats — and the score rides along as an attribute.
        Span span = startIterationSpan(i);
        try (Scope ignored = inScope(span)) {

            Evaluation eval = evaluate(squad.toString(), solution);
            log.info("[EVALUATOR] iteration {} score={} feedback={}",
                    i, eval.score(), eval.feedback());

            if (span != null) {
                span.tag("evaluator.score", String.valueOf(eval.score()));
                span.tag("evaluator.accepted", String.valueOf(eval.score() >= THRESHOLD));
            }

            if (eval.score() >= THRESHOLD) {
                return "Accepted after " + i + " iteration(s) (score "
                        + eval.score() + "):\n" + solution;
            }

            solution = fix(squad.toString(), solution, eval.feedback());

        } finally {
            if (span != null) {
                span.end();
            }
        }
    }
    return "Best effort after " + MAX_ITERATIONS + " iterations:\n" + solution;
}
```

# Evaluator

**The evaluator gets the rules and the squad, and returns a score with feedback.**

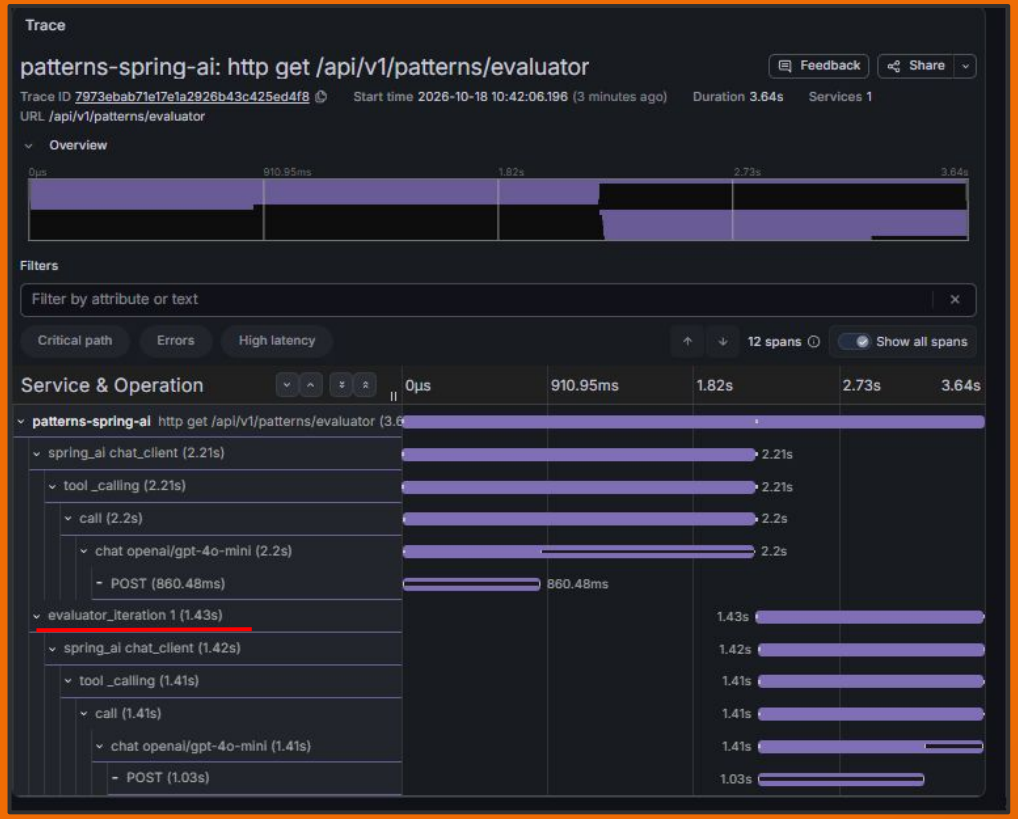
```
private Evaluation evaluate(String squad, String solution) {
    return plainAgent.prompt()
            .user("""
                    Score this lineup from 0.0 to 1.0. Rules, applied in order:
                    - any player NOT in the squad list, or any placeholder such as
                      "TBD" / "not listed" / "could be": score at most 0.3
                    - fewer or more than 11 starters: score at most 0.4
                    - GK, DEF, MID and ATT all covered and the formation coherent: 0.8 or above
                    - reserve 0.9 and above for a lineup that also names shirt numbers
                      for every starter and states the formation explicitly
                    Return the score and concrete feedback."""
                    + "\nSquad: " + squad
                    + "\nLineup: " + solution)
            .call()
            .entity(Evaluation.class);
}
```

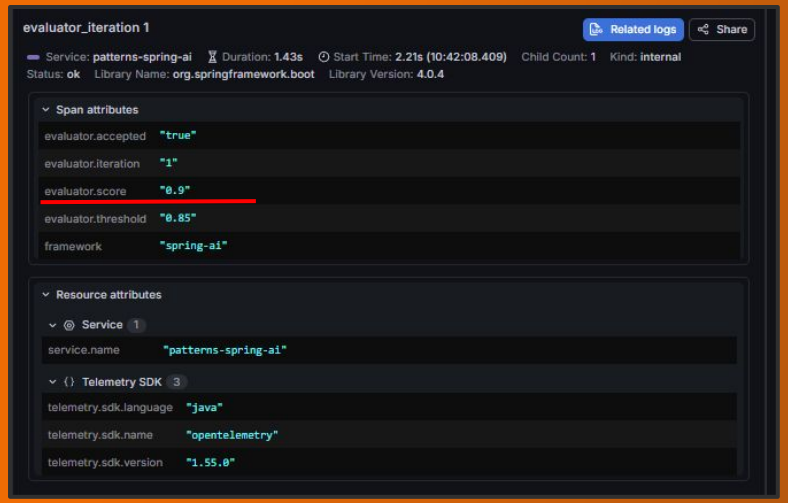
# Fix

**The fixer improves the same lineup, guided by the evaluator’s feedback.**

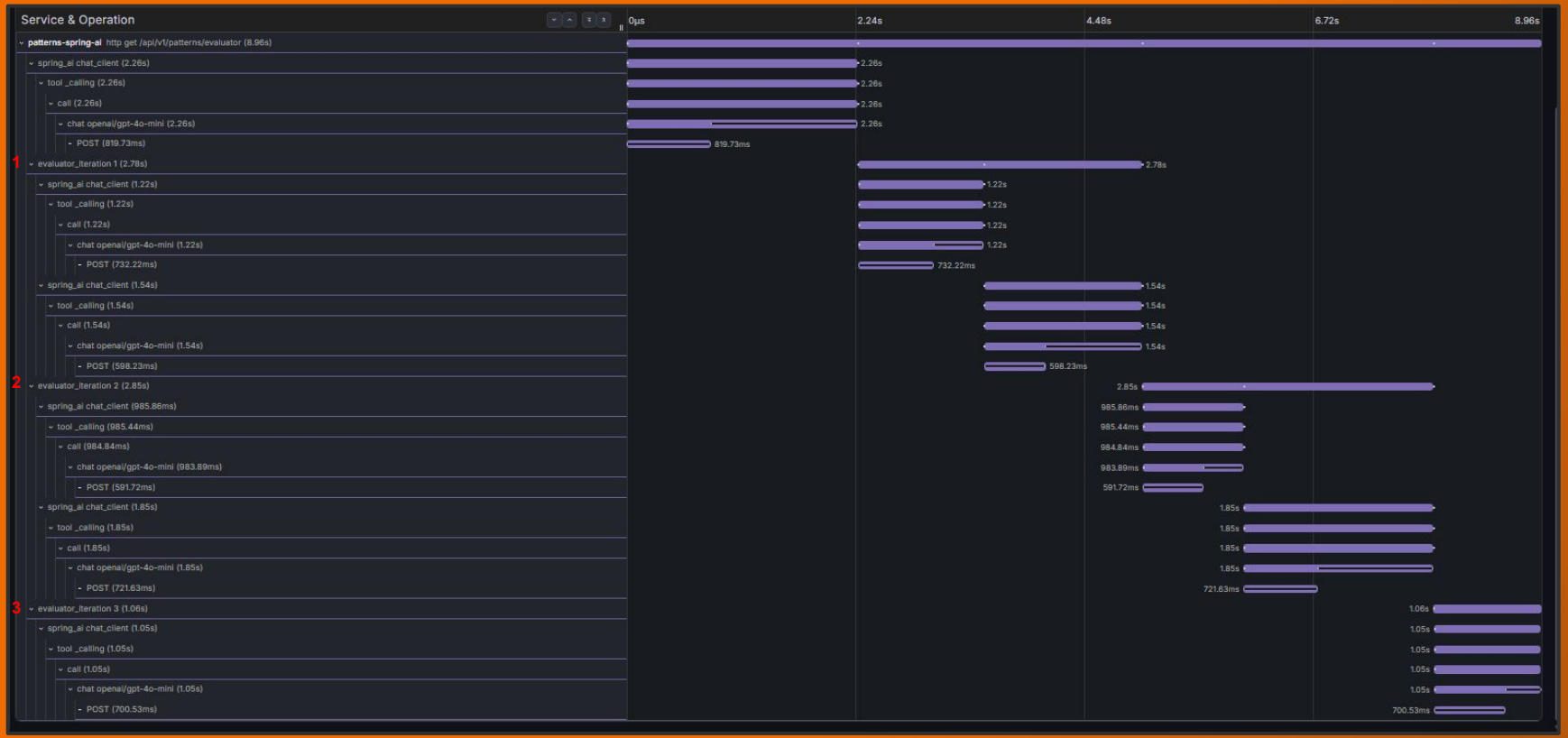
```
/** Improves the EXISTING lineup instead of starting over. */
private String fix(String squad, String solution, String feedback) {
    return plainAgent.prompt()
            .user("Rewrite this lineup so that it has exactly 11 starters, every one "
                    + "of them taken from the squad list (no placeholders, no outsiders), "
                    + "all lines covered, the formation stated explicitly and a shirt "
                    + "number next to every starter. Address this feedback: " + feedback
                    + "\nSquad: " + squad
                    + "\nCurrent lineup: " + solution)
            .call()
            .content();
}
```

# Trace

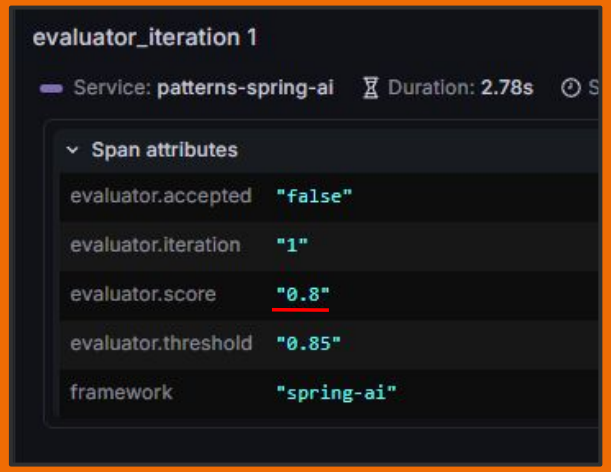

Trace
patterns-spring-ai: http get /api/v1/patterns/evaluator
Feedback
Share
Trace ID 7973ebab71e17e1a2926b43c425ed4f8
Start time 2026-10-18 10:42:06.196 (3 minutes ago)
Duration 3.64s
Services 1
URL /api/v1/patterns/evaluator
Overview
0µs
910.95ms
1.82s
2.73s
3.64s
Filters
Filter by attribute or text
Critical path
Errors
High latency
12 spans
Show all spans
Service & Operation
0µs
910.95ms
1.82s
2.73s
3.64s
patterns-spring-ai http get /api/v1/patterns/evaluator (3.6
spring_ai chat_client (2.21s)
2.21s
tool _calling (2.21s)
2.21s
call (2.2s)
2.2s
chat openai/gpt-4o-mini (2.2s)
2.2s
POST (860.48ms)
860.48ms
evaluator_iteration 1 (1.43s)
1.43s
spring_ai chat_client (1.42s)
1.42s
tool _calling (1.41s)
1.41s
call (1.41s)
1.41s
chat openai/gpt-4o-mini (1.41s)
1.41s
POST (1.03s)
1.03s

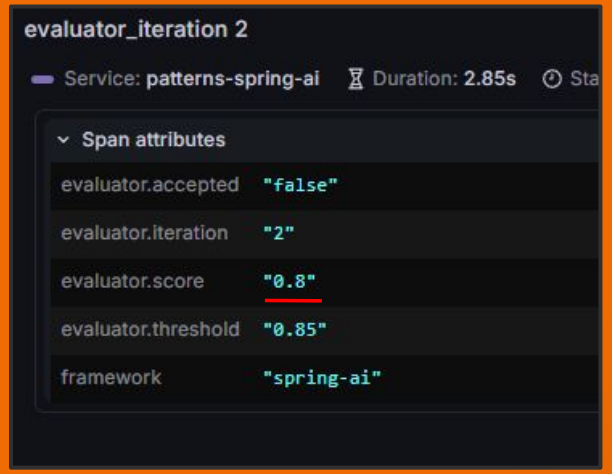

evaluator_iteration 1
Related logs
Share
Service: patterns-spring-ai
Duration: 1.43s
Start Time: 2.21s (10:42:08.409)
Child Count: 1
Kind: internal
Status: ok
Library Name: org.springframework.boot
Library Version: 4.0.4
Span attributes
evaluator.accepted "true"
evaluator.iteration "1"
evaluator.score "0.9"
evaluator.threshold "0.85"
framework "spring-ai"
Resource attributes
Service 1
service.name "patterns-spring-ai"
Telemetry SDK 3
telemetry.sdk.language "java"
telemetry.sdk.name "opentelemetry"
telemetry.sdk.version "1.55.0"

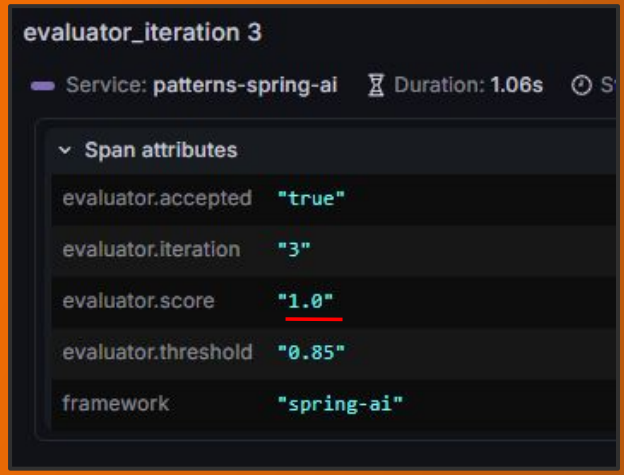
# More accurate trace



# Third time’s the charm



# Evaluator-optimizer → LangChain4j

```
public record Evaluation(double score, String feedback) {}

public interface LineupProposer {
    @Agent("Proposes a starting XI for a squad")
    @UserMessage("Propose a starting XI formation and lineup for AC Milan "
            + "using ONLY players from this list (name and shirt number): {{squadData}}")
    String propose(@V("squadData") String squadData);
}

public interface LineupScorer {
    @Agent("Scores a lineup proposal from 0.0 to 1.0")
    @UserMessage("""
            Score this lineup from 0.0 to 1.0. Rules, applied in order:
            - any player NOT in the squad list, or any placeholder such as
              "TBD" / "not listed" / "could be": score at most 0.3
            - fewer or more than 11 starters: score at most 0.4
            - GK, DEF, MID and ATT all covered and the formation coherent: 0.8 or above
            - reserve 0.9 and above for a lineup that also names shirt numbers
              for every starter and states the formation explicitly
            Return the score and concrete feedback.
            Squad: {{squadData}}
            Lineup: {{solution}}""")
    Evaluation score(@V("squadData") String squadData, @V("solution") String solution);
}
```

# The fixer reads the feedback

```
public interface LineupFixer {
    @Agent("Improves a lineup proposal")
    @UserMessage("Rewrite this lineup so that it has exactly 11 starters, every one "
            + "of them taken from the squad list (no placeholders, no outsiders), "
            + "all lines covered, the formation stated explicitly and a shirt number "
            + "next to every starter. Address this feedback: {{evaluation}}"
            + "\nSquad: {{squadData}}\nCurrent lineup: {{solution}}")
    String fix(@V("squadData") String squadData, @V("solution") String solution,
               @V("evaluation") Evaluation evaluation);
}

/** Typed entry point: proposer once, then the review loop. */
public interface LineupWorkflow {
    @Agent("Produces an accepted starting XI")
    String create(@V("squadData") String squadData);
}
```

# Three agents, three keys

```
public EvaluatorOptimizerPattern(ChatModel chatModel) {
    var proposer = AgenticServices.agentBuilder(LineupProposer.class)
            .chatModel(chatModel)
            .outputKey("solution")
            .build();
    var scorer = AgenticServices.agentBuilder(LineupScorer.class)
            .chatModel(chatModel)
            .outputKey("evaluation")
            .build();
    var fixer = AgenticServices.agentBuilder(LineupFixer.class)
            .chatModel(chatModel)
            .outputKey("solution")
            .build();
```

# The loop is declared

```
    var reviewLoop = AgenticServices.loopBuilder()
            .subAgents(scorer, fixer)
            .maxIterations(MAX_ITERATIONS)
            .exitCondition(scope -> scope.readState("evaluation", new Evaluation(0.0, ""))
                    .score() >= THRESHOLD)
            .build();

    this.workflow = AgenticServices.sequenceBuilder(LineupWorkflow.class)
            .subAgents(proposer, reviewLoop)
            .outputKey("solution")
            .build();
}
```

**Before, the fixer knew that the lineup failed.**

**Now it knows why.**

```
+ public record Evaluation(double score, String feedback) {}

  public interface LineupScorer {
  … the same scoring rules …
-             Return ONLY the number.
+             Return the score and concrete feedback.
-     double score(@V("squadData") String squadData, @V("solution") String solution);
+     Evaluation score(@V("squadData") String squadData, @V("solution") String solution);
  }

  public interface LineupFixer {
  … the same rewrite rules …
-             + "next to every starter. Squad: {{squadData}}\nCurrent lineup: {{solution}}")
+             + "next to every starter. Address this feedback: {{evaluation}}"
+             + "\nSquad: {{squadData}}\nCurrent lineup: {{solution}}")
-     String fix(@V("squadData") String squadData, @V("solution") String solution);
+     String fix(@V("squadData") String squadData, @V("solution") String solution,
+                @V("evaluation") Evaluation evaluation);
  }
```

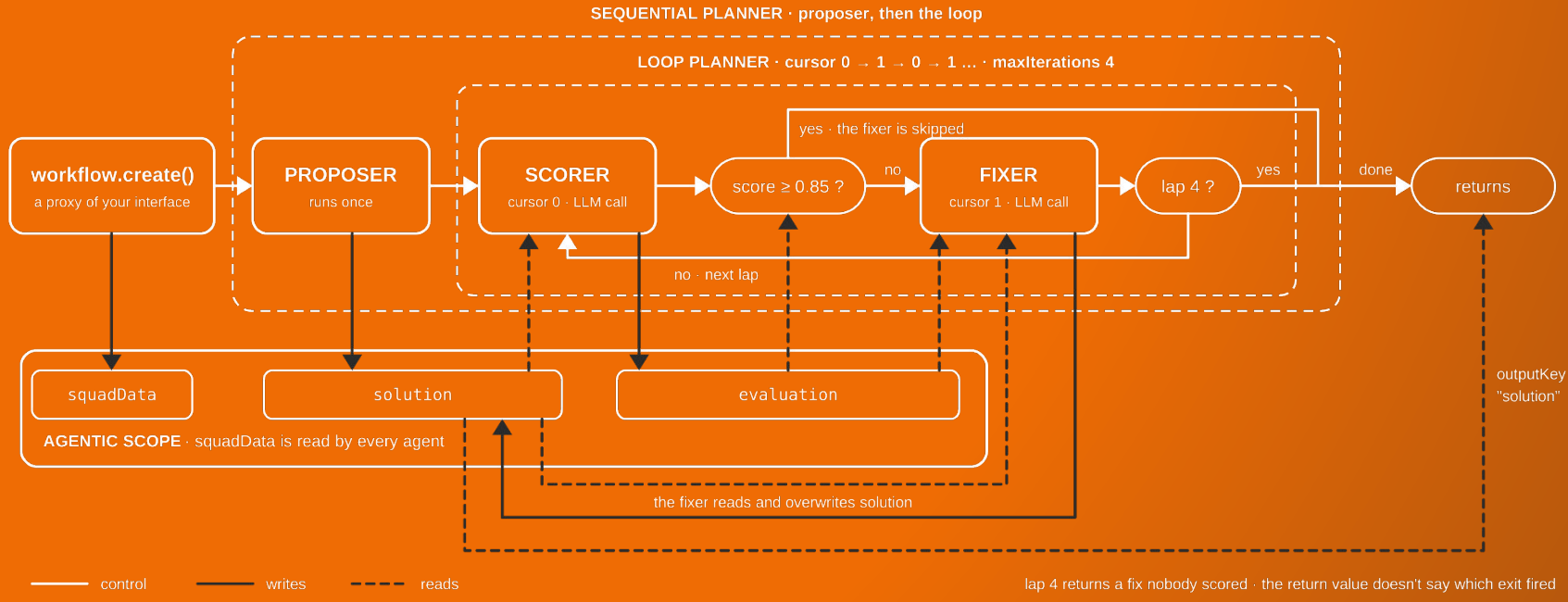
# The exit condition now reads the evaluation.

```
  var scorer = AgenticServices.agentBuilder(LineupScorer.class)
          .chatModel(chatModel)
-         .outputKey("score")
+         .outputKey("evaluation")
          .build();
  …
  var reviewLoop = AgenticServices.loopBuilder()
          .subAgents(scorer, fixer)
          .maxIterations(MAX_ITERATIONS)
-         .exitCondition(scope -> scope.readState("score", 0.0) >= THRESHOLD)
+         .exitCondition(scope -> scope.readState("evaluation", new Evaluation(0.0, ""))
+                 .score() >= THRESHOLD)
          .build();
```

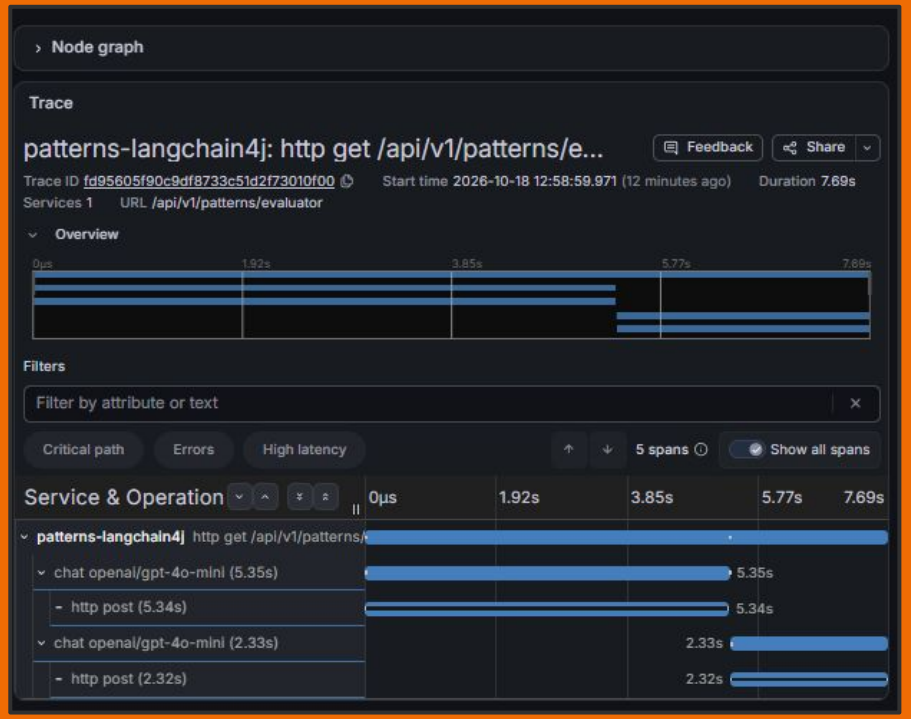
# DSL “loop”

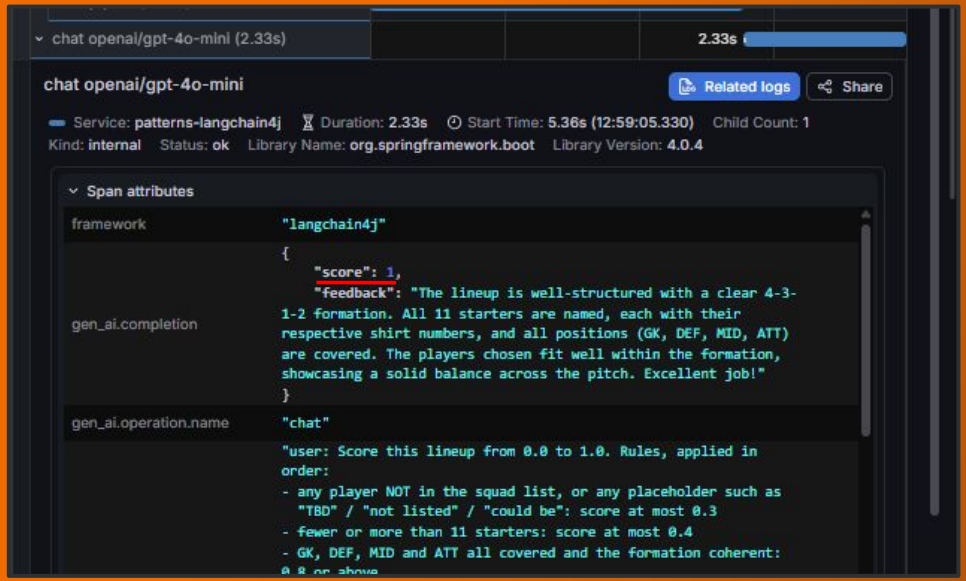
`AgenticServices.loopBuilder()` configures the loop.

A LoopPlanner runs it.



# Trace


Node graph
Trace
patterns-langchain4j: http get /api/v1/patterns/e...
Feedback
Share
Trace ID fd95605f90c9df8733c51d2f73010f00
Start time 2026-10-18 12:58:59.971 (12 minutes ago)
Duration 7.69s
Services 1
URL /api/v1/patterns/evaluator
Overview
0µs
1.92s
3.85s
5.77s
7.69s
Filters
Filter by attribute or text
Critical path
Errors
High latency
5 spans
Show all spans
Service & Operation
patterns-langchain4j http get /api/v1/patterns/
chat openai/gpt-4o-mini (5.35s)
5.35s
http post (5.34s)
5.34s
chat openai/gpt-4o-mini (2.33s)
2.33s
http post (2.32s)
2.32s


chat openai/gpt-4o-mini (2.33s)
2.33s
chat openai/gpt-4o-mini
Related logs
Share
Service: patterns-langchain4j
Duration: 2.33s
Start Time: 5.36s (12:59:05.330)
Child Count: 1
Kind: internal
Status: ok
Library Name: org.springframework.boot
Library Version: 4.0.4
Span attributes
framework
"langchain4j"
gen_ai.completion
{
"score": 1,
"feedback": "The lineup is well-structured with a clear 4-3-1-2 formation. All 11 starters are named, each with their respective shirt numbers, and all positions (GK, DEF, MID, ATT) are covered. The players chosen fit well within the formation, showcasing a solid balance across the pitch. Excellent job!"
}
gen_ai.operation.name
"chat"
"user: Score this lineup from 0.0 to 1.0. Rules, applied in order:
- any player NOT in the squad list, or any placeholder such as "TBD" / "not listed" / "could be": score at most 0.3
- fewer or more than 11 starters: score at most 0.4
- GK, DEF, MID and ATT all covered and the formation coherent:

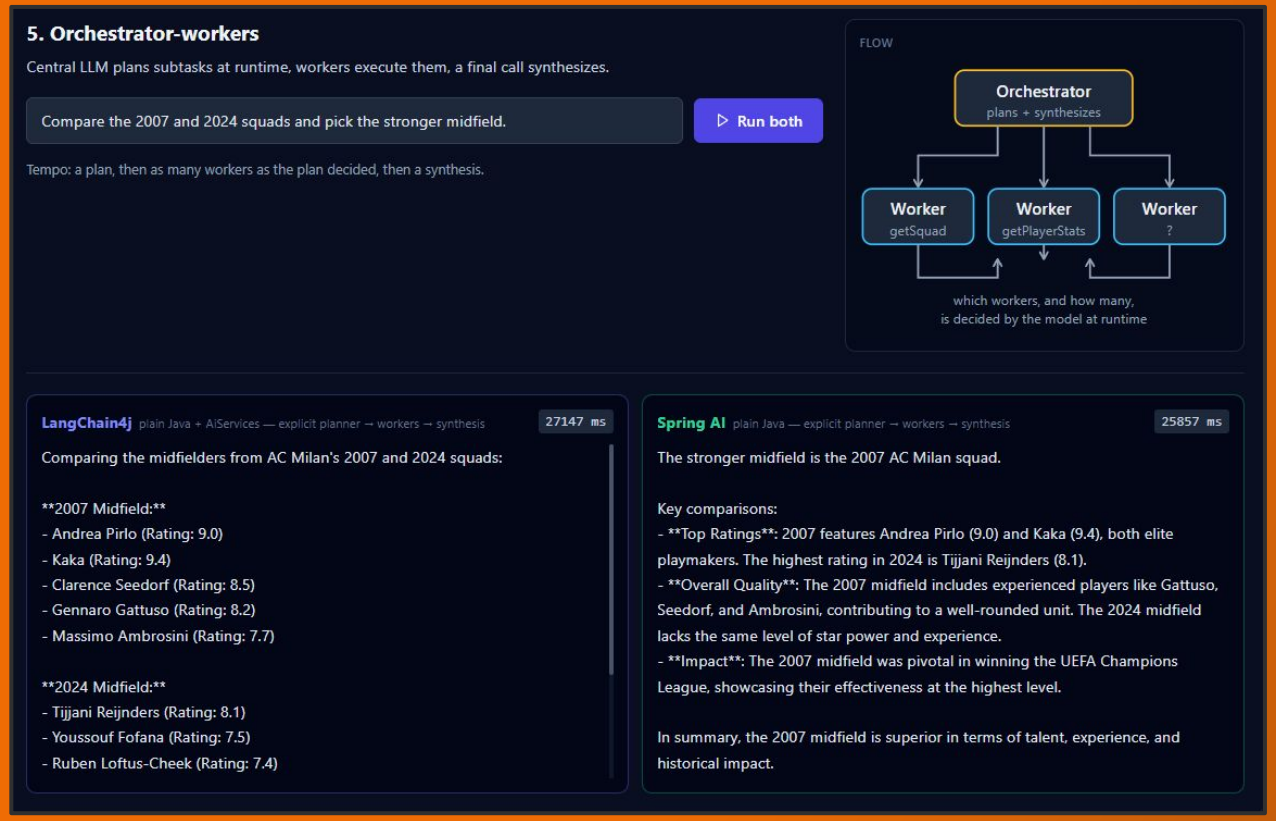
# Orchestrator-workers

The model writes the plan at runtime.

You no longer know the branches.



# Results


5. Orchestrator-workers
Central LLM plans subtasks at runtime, workers execute them, a final call synthesizes.
Compare the 2007 and 2024 squads and pick the stronger midfield.
Run both
Tempo: a plan, then as many workers as the plan decided, then a synthesis.
FLOW
Orchestrator
plans + synthesizes
Worker
getSquad
Worker
getPlayerStats
Worker
?
which workers, and how many,
is decided by the model at runtime
LangChain4j plain Java + AiServices — explicit planner → workers → synthesis
27147 ms
Comparing the midfielders from AC Milan's 2007 and 2024 squads:
**2007 Midfield:**
- Andrea Pirlo (Rating: 9.0)
- Kaka (Rating: 9.4)
- Clarence Seedorf (Rating: 8.5)
- Gennaro Gattuso (Rating: 8.2)
- Massimo Ambrosini (Rating: 7.7)
**2024 Midfield:**
- Tijjani Reijnders (Rating: 8.1)
- Youssouf Fofana (Rating: 7.5)
- Ruben Loftus-Cheek (Rating: 7.4)
Spring AI plain Java — explicit planner → workers → synthesis
25857 ms
The stronger midfield is the 2007 AC Milan squad.
Key comparisons:
- **Top Ratings**: 2007 features Andrea Pirlo (9.0) and Kaka (9.4), both elite playmakers. The highest rating in 2024 is Tijjani Reijnders (8.1).
- **Overall Quality**: The 2007 midfield includes experienced players like Gattuso, Seedorf, and Ambrosini, contributing to a well-rounded unit. The 2024 midfield lacks the same level of star power and experience.
- **Impact**: The 2007 midfield was pivotal in winning the UEFA Champions League, showcasing their effectiveness at the highest level.
In summary, the 2007 midfield is superior in terms of talent, experience, and historical impact.

# Orchestrator-workers → Spring AI

```
public record Plan(List<String> subtasks) {}

public String run(String task) {
    // Phase 1 — the orchestrator decomposes the task (typed plan)
    Plan plan = plainAgent.prompt()
            .user("Decompose this AC Milan analysis task into 2-4 concrete, "
                    + "independent data-gathering subtasks (each answerable with "
                    + "squad/transfer/stats/rumor data): " + task)
            .call()
            .entity(Plan.class);
    log.info("[ORCHESTRATOR] plan: {}", plan.subtasks());
```

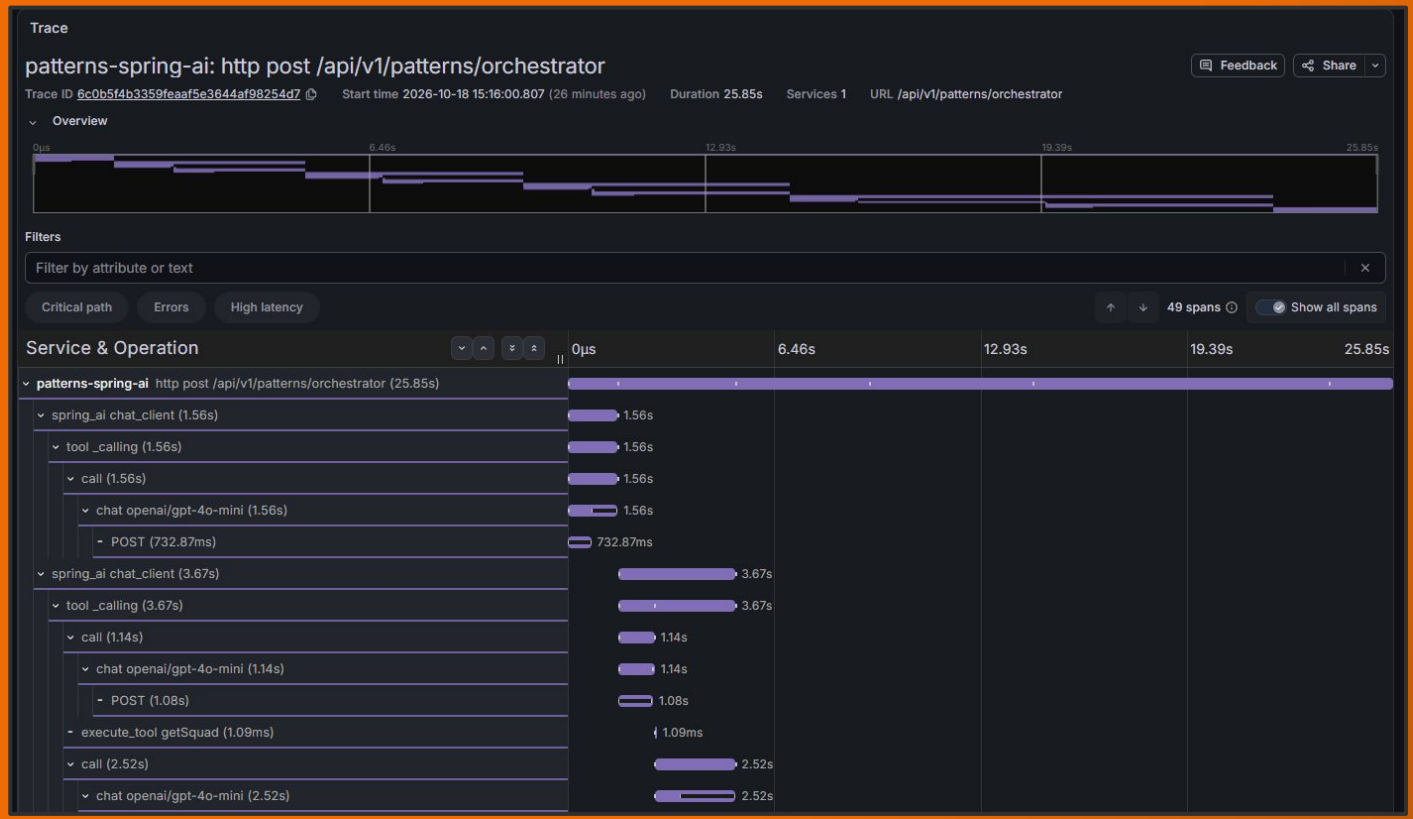
# One worker call per subtask

```
// Phase 2 — workers execute sub-tasks (tools attached)
String workerResults = plan.subtasks().stream()
        .map(subtask -> {
            log.info("[ORCHESTRATOR] worker -> {}", subtask);
            String result = milanAgent.prompt()           // the client with the tools
                    .user(subtask)
                    .call()
                    .content();
            return "- " + subtask + "\n  " + result;
        })
        .collect(Collectors.joining("\n"));
```

# One last call merges the results

```
    // Phase 3 — synthesis
    return plainAgent.prompt()
            .user("Synthesize a final answer to the task: '" + task
                    + "' from these worker results:\n" + workerResults)
            .call()
            .content();
}
```

# Trace


Trace
patterns-spring-ai: http post /api/v1/patterns/orchestrator
Feedback
Share
Trace ID 6c0b5f4b3359feaaf5e3644af98254d7
Start time 2026-10-18 15:16:00.807 (26 minutes ago)
Duration 25.85s
Services 1
URL /api/v1/patterns/orchestrator
Overview
0µs
6.46s
12.93s
19.39s
25.85s
Filters
Filter by attribute or text
Critical path
Errors
High latency
49 spans
Show all spans
Service & Operation
0µs
6.46s
12.93s
19.39s
25.85s
patterns-spring-ai http post /api/v1/patterns/orchestrator (25.85s)
spring_ai chat_client (1.56s)
1.56s
tool _calling (1.56s)
1.56s
call (1.56s)
1.56s
chat openai/gpt-4o-mini (1.56s)
1.56s
POST (732.87ms)
732.87ms
spring_ai chat_client (3.67s)
3.67s
tool _calling (3.67s)
3.67s
call (1.14s)
1.14s
chat openai/gpt-4o-mini (1.14s)
1.14s
POST (1.08s)
1.08s
execute_tool getSquad (1.09ms)
1.09ms
call (2.52s)
2.52s
chat openai/gpt-4o-mini (2.52s)
2.52s

# Orchestrator-workers → LangChain4j

```
public record Plan(List<String> subtasks) {}

interface Planner {
    @SystemMessage("You are a concise football analyst. Answer briefly.")
    @UserMessage("Decompose this AC Milan analysis task into 2-4 concrete, "
            + "independent data-gathering subtasks (each answerable with "
            + "squad/transfer/stats/rumor data): {{task}}")
    Plan plan(@V("task") String task);
}

interface Worker {
    @SystemMessage("""
            You are an AC Milan data analyst.
            Use the available tools to fetch squads, transfers,
            player stats and (only when explicitly asked) secret rumors.
            Never invent data — if a tool returns nothing, say so.
            """)
    String work(@UserMessage String subtask);
}
```

# Three AI services, one job each

```
interface Synthesizer {
    @SystemMessage("You are a concise football analyst. Answer briefly.")
    @UserMessage("Synthesize a final answer to the task: '{{task}}' "
            + "from these worker results:\n{{workerResults}}")
    String synthesize(@V("task") String task, @V("workerResults") String workerResults);
}

public OrchestratorWorkersPattern(ChatModel chatModel, ToolProvider milanToolProvider) {
    this.planner = AiServices.builder(Planner.class)
            .chatModel(chatModel)
            .build();                                    // no tools: it only plans
    this.worker = AiServices.builder(Worker.class)
            .chatModel(chatModel)
            .toolProvider(milanToolProvider)             // only the workers get tools
            .build();
    this.synthesizer = AiServices.builder(Synthesizer.class)
            .chatModel(chatModel)
            .build();
}
```

# Plan, dispatch, synthesize

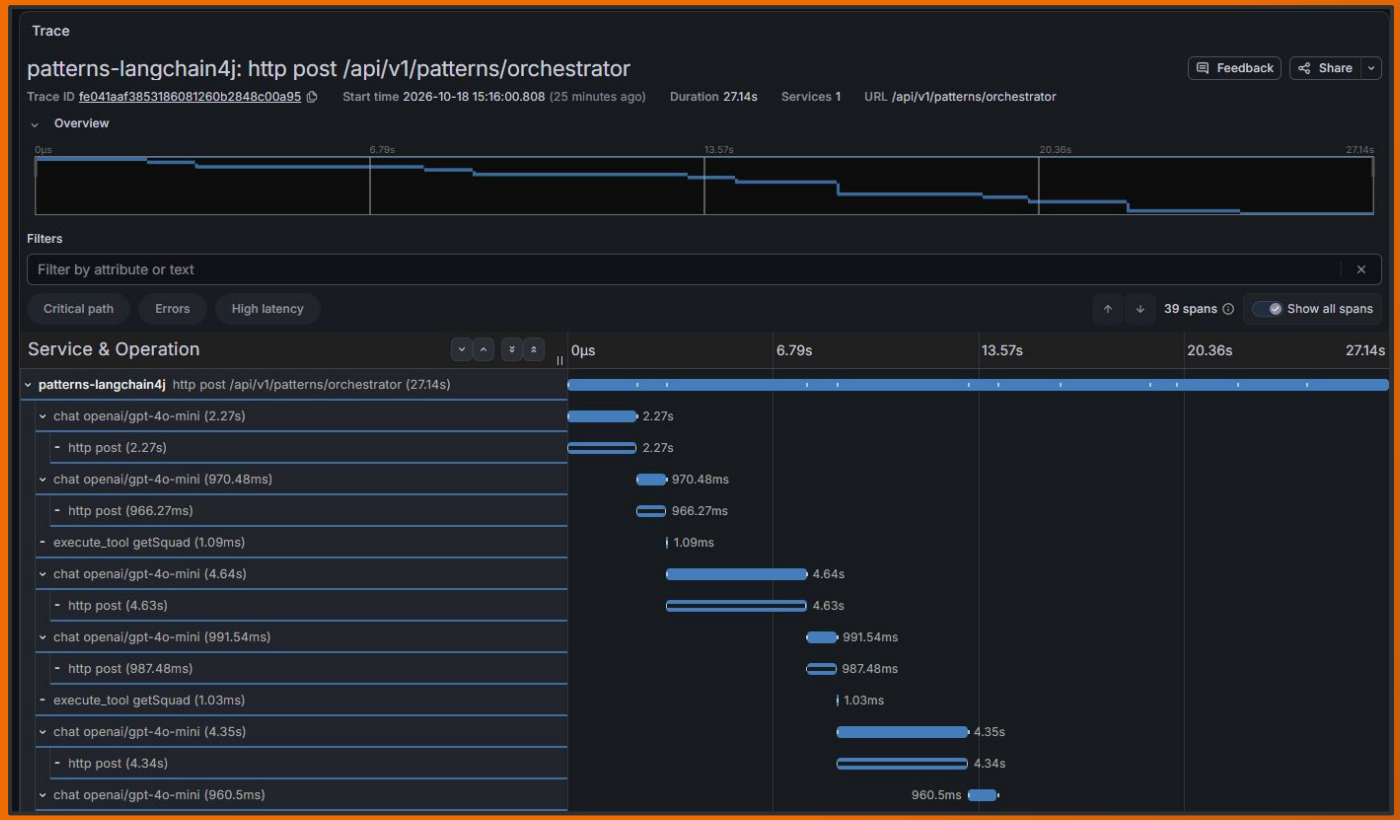
```
public String run(String task) {
    log.info("[ORCHESTRATOR] task: {}", task);

    // Phase 1 — the orchestrator decomposes the task (typed plan)
    Plan plan = planner.plan(task);
    log.info("[ORCHESTRATOR] plan: {}", plan.subtasks());

    // Phase 2 — workers execute sub-tasks (tools attached)
    String workerResults = plan.subtasks().stream()
            .map(subtask -> {
                log.info("[ORCHESTRATOR] worker -> {}", subtask);
                return "- " + subtask + "\n  " + worker.work(subtask);
            })
            .collect(Collectors.joining("\n"));

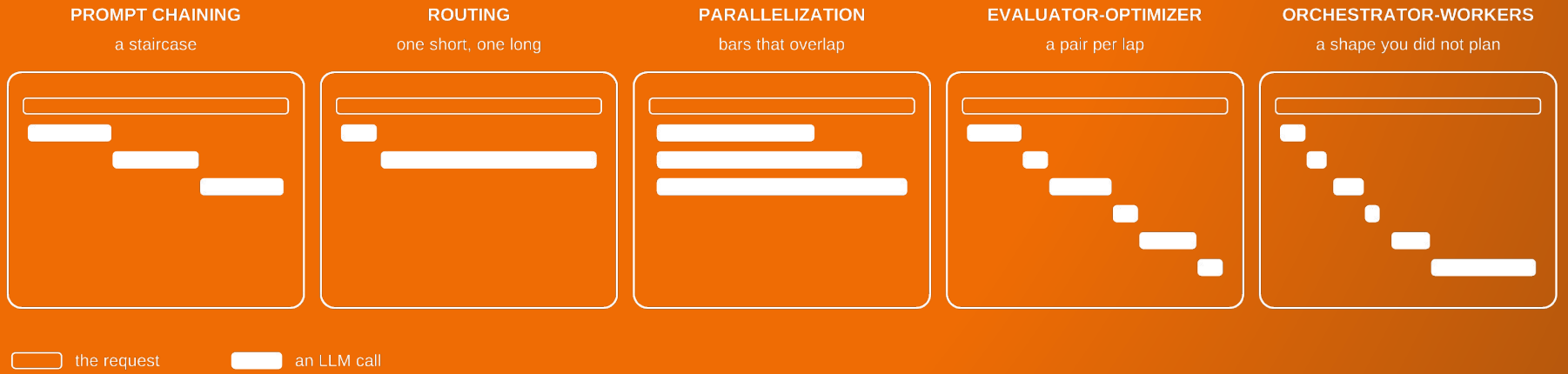
    // Phase 3 — synthesis
    return synthesizer.synthesize(task, workerResults);
}
```

# Trace

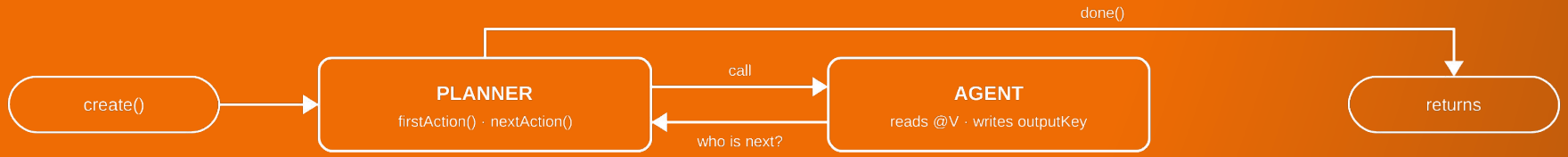

Trace
patterns-langchain4j: http post /api/v1/patterns/orchestrator
Feedback
Share
Trace ID fe041aaf3853186081260b2848c00a95
Start time 2026-10-18 15:16:00.808 (25 minutes ago)
Duration 27.14s
Services 1
URL /api/v1/patterns/orchestrator
Overview
0µs
6.79s
13.57s
20.36s
27.14s
Filters
Filter by attribute or text
Critical path
Errors
High latency
39 spans
Show all spans
Service & Operation
patterns-langchain4j http post /api/v1/patterns/orchestrator (27.14s)
chat openai/gpt-4o-mini (2.27s)
2.27s
http post (2.27s)
2.27s
chat openai/gpt-4o-mini (970.48ms)
970.48ms
http post (966.27ms)
966.27ms
execute_tool getSquad (1.09ms)
1.09ms
chat openai/gpt-4o-mini (4.64s)
4.64s
http post (4.63s)
4.63s
chat openai/gpt-4o-mini (991.54ms)
991.54ms
http post (987.48ms)
987.48ms
execute_tool getSquad (1.03ms)
1.03ms
chat openai/gpt-4o-mini (4.35s)
4.35s
http post (4.34s)
4.34s
chat openai/gpt-4o-mini (960.5ms)
960.5ms

# Every pattern leaves its own waterfall

Open the trace in Tempo and the shape tells you which pattern ran.



# Same loop, three planners




| BUILDER | PLANNER | PICKS THE NEXT AGENT | STOPS WHEN |
|---|---|---|---|
| `sequenceBuilder()` | `SequentialPlanner` | the next one in the list | after the last one |
| `conditionalBuilder()` | `ConditionalPlanner` | every branch whose condition holds | after that one pass |
| `loopBuilder()` | `LoopPlanner` | the next one, wrapping around | exit condition or maxIterations |

# Two ways to write a workflow

**LangChain4j declares it.**

**Spring AI lets you write it.**

| | LANGCHAIN4J<br>declares it | SPRING AI<br>lets you write it |
|---|---|---|
| ONE STEP | `agentBuilder(…).outputKey("analysis")` | `chatClient.prompt().user(…).call()` |
| PROMPT CHAINING | `sequenceBuilder().subAgents(a, b, c)` | three calls, one after another |
| ROUTING | `conditionalBuilder() + predicates` | `switch (route) { … }` |
| PARALLELIZATION | `CompletableFuture`<br>parallelMapperBuilder() exists, not used | `CompletableFuture` |
| EVALUATOR-OPTIMIZER | `loopBuilder().exitCondition(…)` | `for (…) { if (…) return; }` |
| ORCHESTRATOR-WORKERS | plan → workers → synthesis<br>supervisorBuilder() exists, not used | plan → workers → synthesis |
| STATE | `AgenticScope · outputKey / @V` | local variables |

the DSL does the work  plain Java on both sides

# See you in part Ⅲ next week!

